Jour 1 : diagnostic et début de Nx

Copier un code : cliquer dans la case blanche à droite du titre du bloc, puis **Cmd+A** et **Cmd+C** (sous Windows : **Ctrl+A** et **Ctrl+C**). Le code copié est exact, avec son indentation.

Plan de la journée

Horaire	Partie
9 h 00 – 10 h 00	Introduction de la formation
10 h 00 – 10 h 45	Visite de ShopFlow, l'application de départ
10 h 45 – 11 h 00	pause
11 h 00 – 12 h 30, 13 h 30 – 14 h 30	Séquence 2 : diagnostiquer ShopFlow
12 h 30 – 13 h 30	déjeuner
14 h 30 – 15 h 15	Séquence 3 : Nx dans ShopFlow (première partie)
15 h 15 – 15 h 30	pause
15 h 30 – 15 h 40	TP final : remettre Cuisto en état
15 h 40 – 17 h 00	TP Cuisto, jour 1

9 h 00 – 10 h 00

Introduction de la formation

Au programme de l'introduction (1 h)

- 5 min : présentation du formateur
- 10 min : tour de table
- 10 min : le programme et l'organisation des journées
- 15 min : test de niveau
- 5 min : vérification des postes
- 5 min : récupération des deux applications
- 10 min : lancement et test des deux applications

Pas de code tant que les présentations et le programme ne sont pas faits : on fait connaissance, on explique où l'on va, puis seulement on prépare les postes et les applications.

3. Le programme et l'organisation (10 min)

Ce qu'on va faire : partir d'une boutique en ligne qui fonctionne mais qui a des défauts réels, ShopFlow, la mesurer, en dresser le cahier des charges, puis la corriger module par module. Chaque après-midi, chacun applique seul les mêmes notions à une seconde application, Cuisto, dont le code est différent : la réflexion est la même, la solution n'est pas à recopier.

Horaire	Chaque journée
9 h 00 – 15 h 15	Les modules du jour sur ShopFlow, ensemble : vous faites en même temps que le formateur
15 h 15 – 15 h 30	Pause
15 h 30 – 17 h 00	TP Cuisto : vous appliquez seuls les modules du jour ; le jour 3, remise et évaluation à 16 h 45

Jour	Modules du programme
1	Module 1 : rappels, Nx, architecture, chargement à la demande, documentation, ChangeDetectorRef
2	Module 2 : Zone.js, RxJS, guards et resolver ; module 3 : signaux ; module 4 : standalone
3	Module 5 : internationalisation ; module 6 : formulaires ; module 7 : tests ; module 8 : rendu côté serveur

Le programme a été écrit à l'époque d'Angular 14-15 (2022). Nous travaillons en Angular 22 : certains points existent toujours, d'autres ont été remplacés. On travaille alors avec le remplaçant, en expliquant ce qu'il remplace, parce qu'on rencontre encore l'ancien code en entreprise.

Dans le programme	Aujourd'hui (Angular 22)
Protractor (tests de bout en bout)	Abandonné en 2023 : Playwright
Angular Universal : `ServerModule`, `@nguniversal/express-engine`	`@angular/ssr`, avec l'hydratation
NgModules, passage au standalone	Standalone par défaut depuis Angular 19
Zone.js, exécution hors zone	Les nouveaux projets s'en passent depuis Angular 21
Karma	Toujours utilisable ; Vitest est le choix par défaut depuis Angular 21
Guards et resolvers en classes	Fonctions `CanActivateFn`, `CanActivateChildFn`, `ResolveFn`
Non cité : `*ngIf`, `*ngFor`	`@if`, `@for`, `@switch` depuis Angular 17

4. Test de niveau (15 min)

Cette formation est une formation avancée : elle suppose les bases d'Angular. Un test de 20 questions, corrigé automatiquement, permet de savoir tout de suite qui est à quel niveau.

Lien du test : https://tally.so/r/bZV1p6

Pendant que les apprenants répondent, ouvrir l'onglet des réponses du formulaire dans Tally : une ligne par apprenant, avec son prénom, ses réponses et son score.

Score sur 20	Ce qu'on fait
15 et plus	Les bases sont acquises
10 à 14	Noter les notions manquées ; les reprendre pendant le rappel et la visite de ShopFlow
Moins de 10	Niveau d'une formation avancée non atteint ; suivre l'apprenant de près et en parler au responsable pédagogique

5. Vérification des postes (5 min)

Chaque apprenant ouvre un terminal et vérifie :

Terminal — n'importe quel dossier `node -v`

```
node -v
npm -v
git --version
```

> **Version de Node trop ancienne**
>
> Angular 22.2 exige **Node 24.15 ou plus récent** (ou 22.22.3 et plus si l'on reste en Node 22). Avec une version plus ancienne, `npm install` affiche des avertissements « EBADENGINE » et Angular CLI refuse de démarrer (« The Angular CLI requires a minimum Node.js version… »).
>
> Mettre à jour : installer la version **LTS** depuis https://nodejs.org, puis fermer et rouvrir le terminal. Avec nvm (macOS, Linux) : `nvm install 24` puis `nvm use 24`. Vérifier ensuite avec `node -v`.

Outils à installer s'ils manquent : **Google Chrome** avec l'extension **Angular DevTools** ; **Visual Studio Code** avec l'extension **Angular Language Service**.

6. Récupérer les deux applications (5 min)

- **ShopFlow**, l'application étudiée ensemble : https://github.com/adamfrance/cours-angular-avance-shopflow-depart
- **Cuisto**, l'application des TP de fin de journée : https://github.com/adamfrance/cours-angular-avance-cuisto-app-exercices

On crée d'abord un dossier de travail pour la formation, puis on y clone les deux applications :

Terminal — dans le dossier où l'on range ses projets (par exemple le Bureau) `mkdir formation-angular`

```
mkdir formation-angular
cd formation-angular
git clone https://github.com/adamfrance/cours-angular-avance-shopflow-depart.git shopflow
git clone https://github.com/adamfrance/cours-angular-avance-cuisto-app-exercices.git cuisto
```

7. Lancer et tester les deux applications (10 min)

Les deux applications tournent en même temps, une par terminal. `npm install` prend une à trois minutes selon le réseau : lancer les deux installations, puis vérifier pendant ce temps que tout le monde a bien cloné.

Terminal 1 — ShopFlow, dossier formation-angular/shopflow (http://localhost:4200) `cd formation-angular/shopflow`

```
cd formation-angular/shopflow
npm install
npm start
```

Terminal 2 — Cuisto, dossier formation-angular/cuisto (http://localhost:4201) `cd formation-angular/cuisto`

```
cd formation-angular/cuisto
npm install
npm start -- --port 4201
```

Application	Vérification	Attendu
ShopFlow	Ouvrir http://localhost:4200	Le catalogue affiche « 194 produits »
ShopFlow	Se connecter avec `emilys` / `emilyspass`	« Bonjour Emily » dans l'en-tête
Cuisto	Ouvrir http://localhost:4201	La liste affiche « 50 recette(s) »
Cuisto	Ouvrir une recette	La fiche s'affiche avec ses ingrédients
Les deux	F12, onglet Console	Aucune erreur en rouge

> **Si une application ne démarre pas**
>
> Vérifier la version de Node (24.15 ou plus récent), relancer `npm install`, vérifier que le port n'est pas déjà utilisé. Ne pas commencer le cours tant que les deux applications ne tournent pas chez tout le monde : un apprenant bloqué ici le reste toute la journée.

10 h 00 – 10 h 45

Visite de ShopFlow, l’application de départ

Rappel : écrire un template aujourd’hui

Avant la visite, on montre la différence entre l’écriture que certains apprenants connaissent et celle de ShopFlow.

Ancienne et nouvelle écriture d’un template

```
<!-- Avant Angular 17 -->
<p *ngIf="produits.length > 0; else vide">{{ produits.length }} produits</p>
<ng-template #vide><p>Aucun produit</p></ng-template>
<li *ngFor="let p of produits; trackBy: parId">{{ p.title }}</li>

<!-- Angular 17 et suivants -->
@if (produits.length > 0) {
  <p>{{ produits.length }} produits</p>
} @else {
  <p>Aucun produit</p>
}
@for (p of produits; track p.id) {
  <li>{{ p.title }}</li>
}
```

Au programme (45 min)

1. **Lancer ShopFlow (15 min)** : récupérer le projet, installer, démarrer.
2. **Le parcours d’un client et d’un administrateur (15 min)** : on utilise l’application comme un vrai utilisateur, page par page.
3. **L’organisation du code (15 min)** : le projet et ses dossiers, le démarrage, l’arbre des composants, les routes, les services.
4. **Six interactions expliquées (25 min)** : le catalogue, le panier, la connexion, mes commandes, la commande, le tableau de bord.
5. **Ce qu’on a remarqué (5 min)** : la liste des constats, point de départ du diagnostic de la séquence suivante.

Pendant toute la visite, le formateur projette l’application et le code ; les apprenants ont l’application ouverte chez eux et répondent aux questions. On ne corrige rien aujourd’hui : on observe et on note.

Remarque métier

Découvrir une application qu’on n’a pas écrite est la situation la plus fréquente en entreprise : arrivée dans une équipe, reprise d’un projet par un prestataire, tierce maintenance applicative (TMA). La méthode est toujours la même : l’utiliser comme un client, repérer les fichiers qui portent chaque fonctionnalité, suivre le trajet des données, et noter tout ce qui surprend. C’est exactement ce que nous faisons ici.

1. Lancer ShopFlow

ShopFlow est l'application de départ de la formation : une boutique en ligne écrite en Angular 22, dans un projet Angular CLI classique : composants standalone, `@if` et `@for`, `inject()`, sans Zone.js. Nx n'y est pas encore : on l'installera au module 1, quand le diagnostic aura montré pourquoi on en a besoin. Elle fonctionne, mais elle contient les défauts qu'on rencontre dans les vraies applications ; les trois jours de formation consistent à les trouver puis à les corriger, module par module.

Les données viennent de l'API publique DummyJSON : 194 produits, de vrais comptes utilisateurs avec connexion par jeton, et des paniers qui servent de commandes. Rien à installer côté serveur. Les données de l'API sont en anglais ; l'interface est en français.

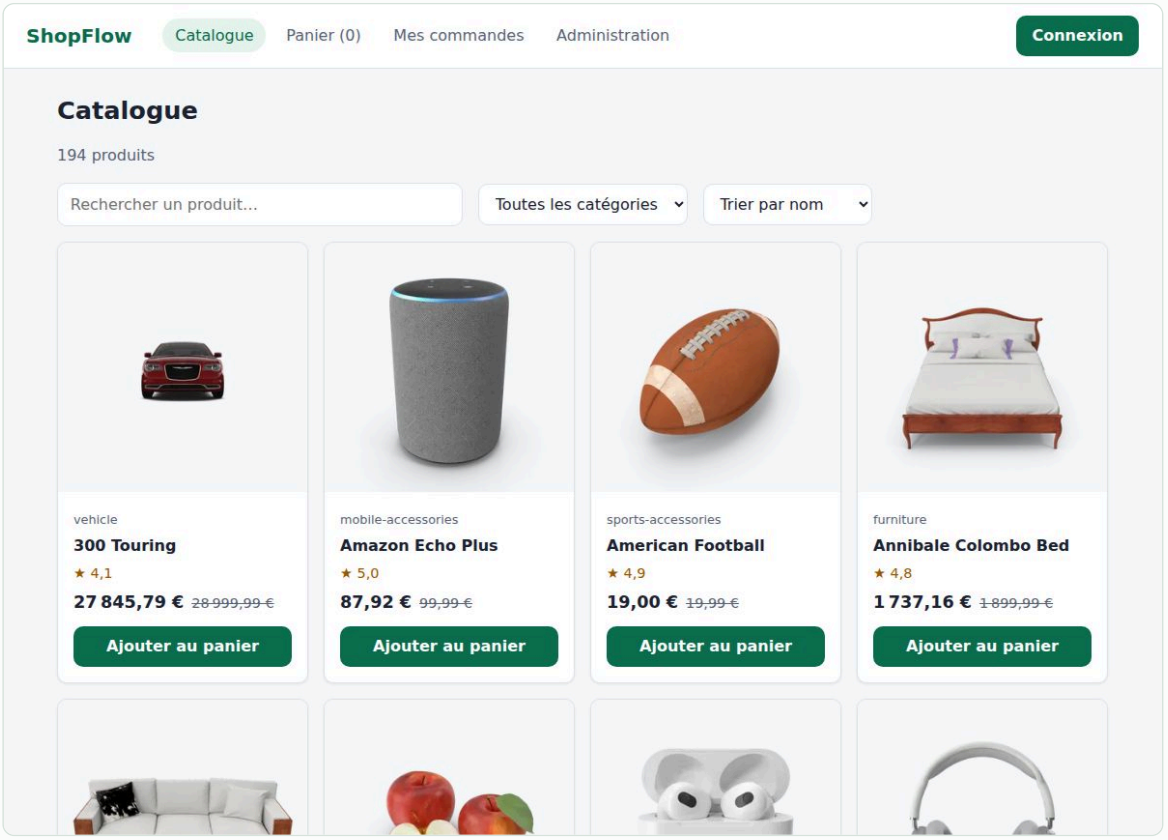
ShopFlow a été cloné et lancé pendant l'introduction : il tourne dans le terminal 1 (dossier `formation-angular/shopflow`, commande `npm start`) sur `http://localhost:4200`.

L'application s'ouvre sur http://localhost:4200. Compte de démonstration à donner aux apprenants : identifiant `emilys`, mot de passe `emilyspass` (Emily Johnson, rôle administrateur dans l'API).

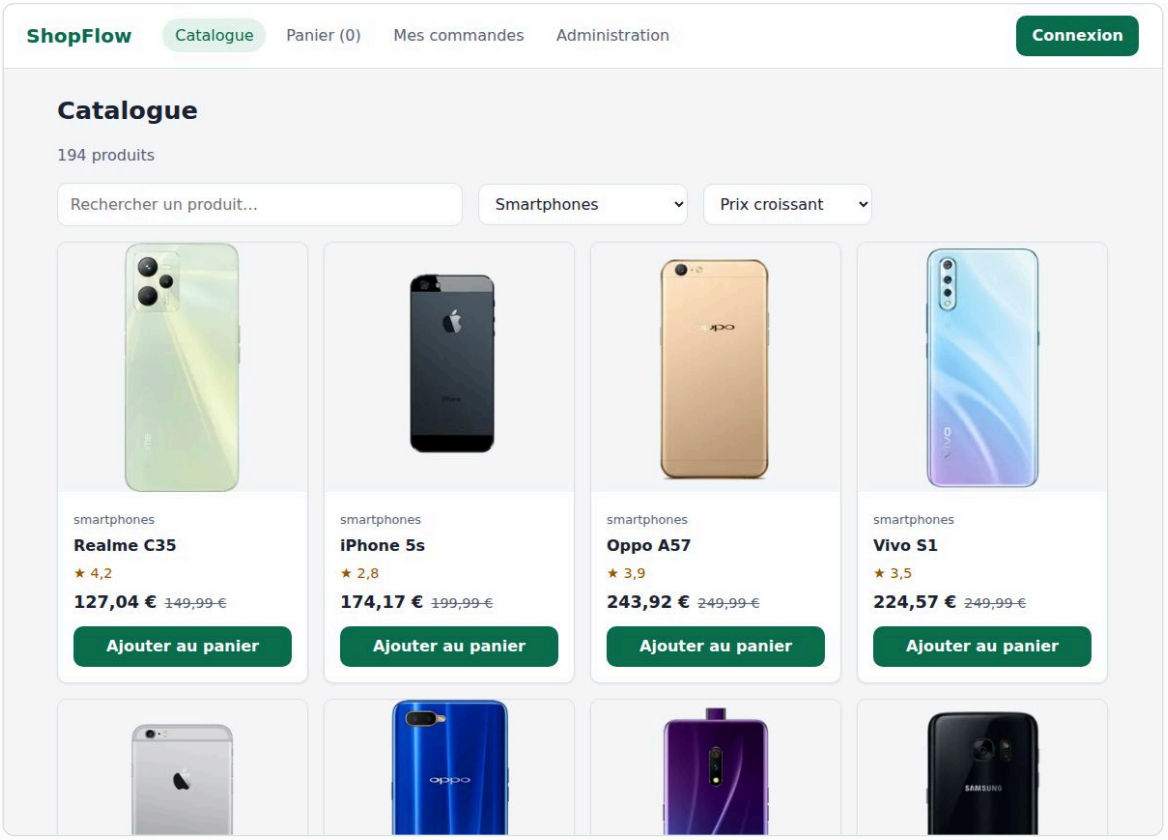
2. Le parcours d'un client et d'un administrateur

Faites le parcours en direct, dans cet ordre, en demandant aux apprenants de le refaire chez eux. Ouvrez les outils de développement (F12), onglet **Réseau**, avant de commencer : on y verra les requêtes partir.

Le catalogue



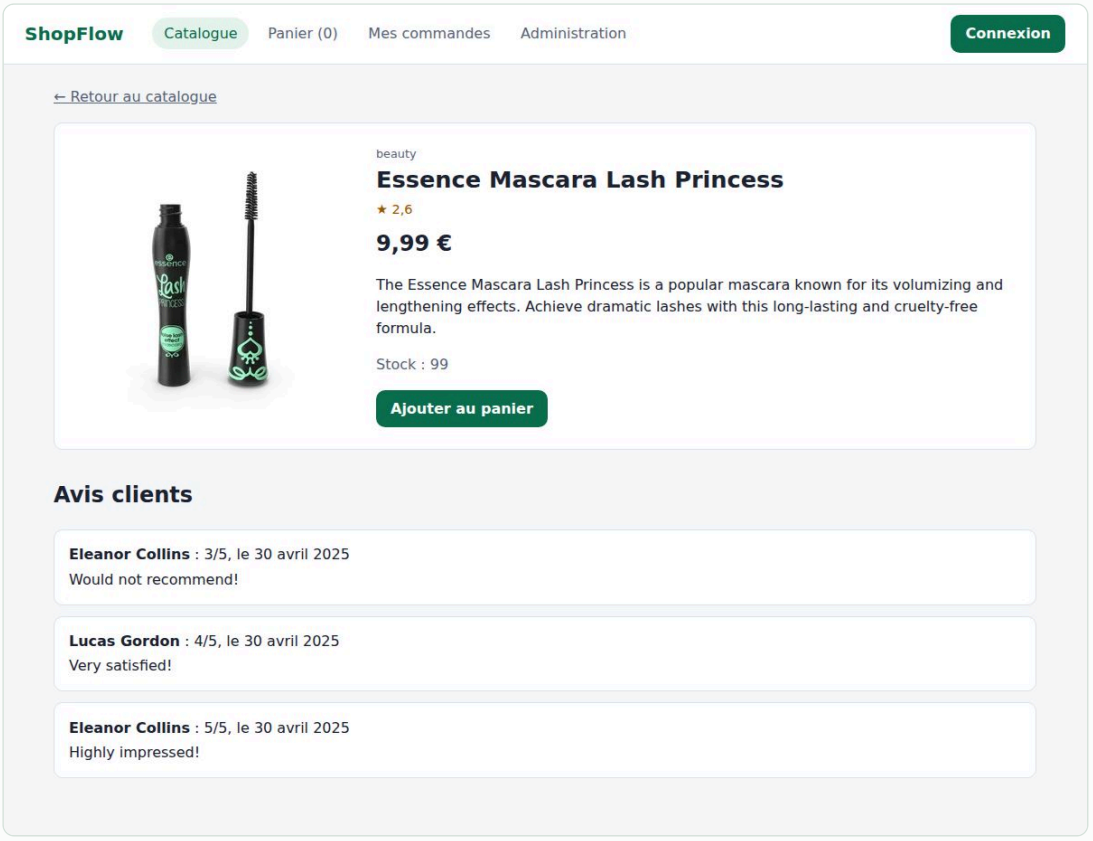
/catalogue : les 194 produits, une recherche par nom, un filtre par catégorie, un tri. Le prix affiché tient compte de la remise.



Catégorie « Smartphones » et tri par prix croissant : le filtrage se fait dans le navigateur, sans nouvelle requête.

À montrer dans l'onglet Réseau : une requête `products?limit=0` d'environ 140 ko, une requête `categories`, puis **194 images** téléchargées d'un coup, même celles tout en bas de la page.

La fiche produit



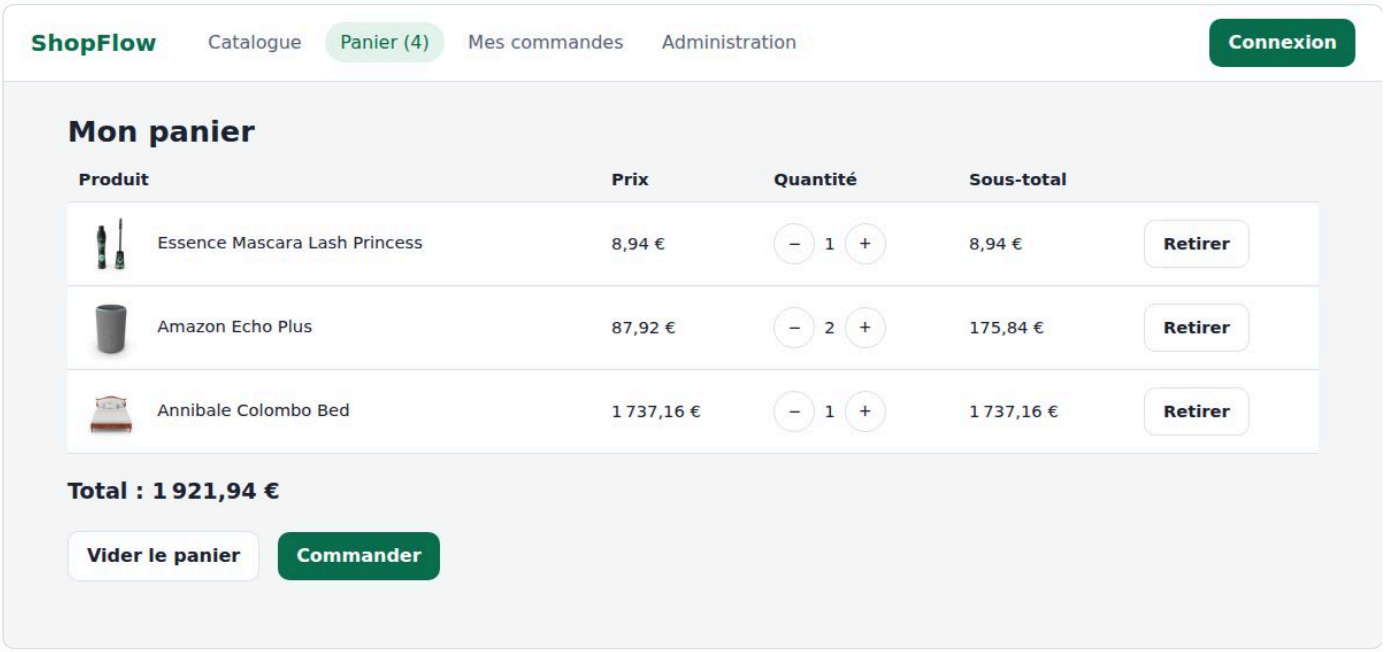
/catalogue/1 : photo, description, stock, avis clients et bouton « Ajouter au panier ».

Faites remarquer le prix : 9,99 € sur la fiche, alors que la carte du catalogue affichait 8,94 €. La fiche oublie la remise : c'est un vrai bogue, que des tests (module 7) auraient détecté.

Le panier



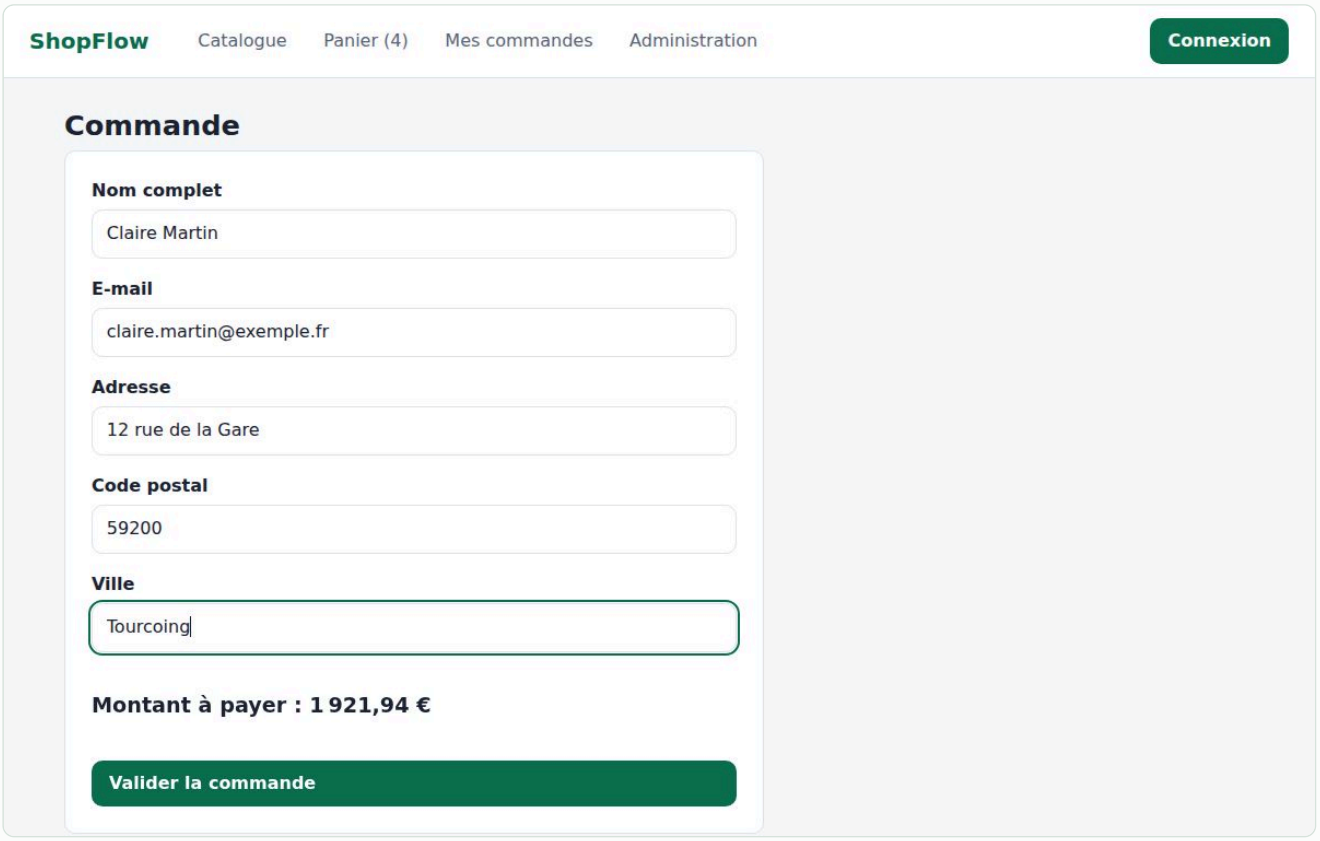
Après trois clics sur « Ajouter au panier » : le compteur de l'en-tête passe à 4, sans rechargement.



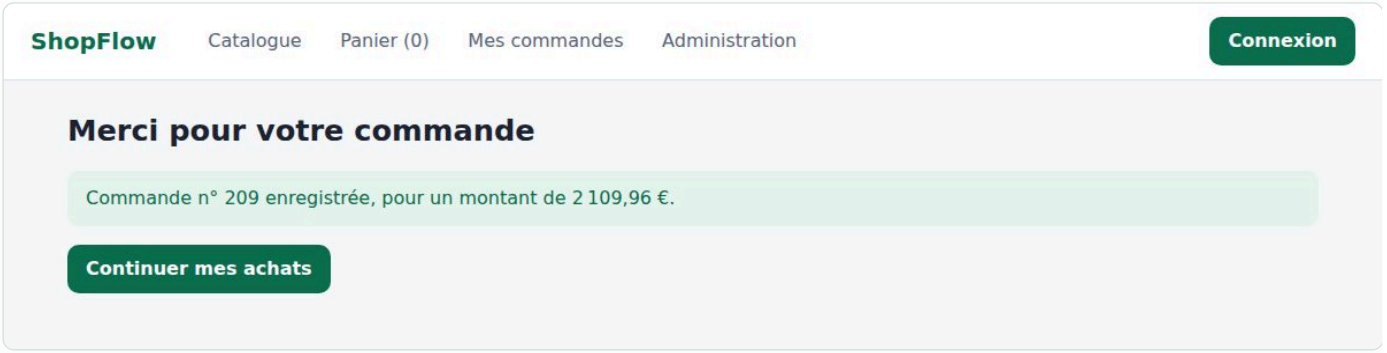
/panier : quantités modifiables, sous-totaux, total, « Vider le panier » et « Commander ».

À montrer : rechargez la page (F5), le panier est toujours là. Onglet **Application** des outils de développement, rubrique Local Storage : la clé `shopflow-panier` contient le panier en JSON.

La commande



/commande : nom, e-mail, adresse, code postal et ville, puis le montant à payer.



Après « Valider la commande » : l'API renvoie la commande créée (n° 209), le panier est vidé.

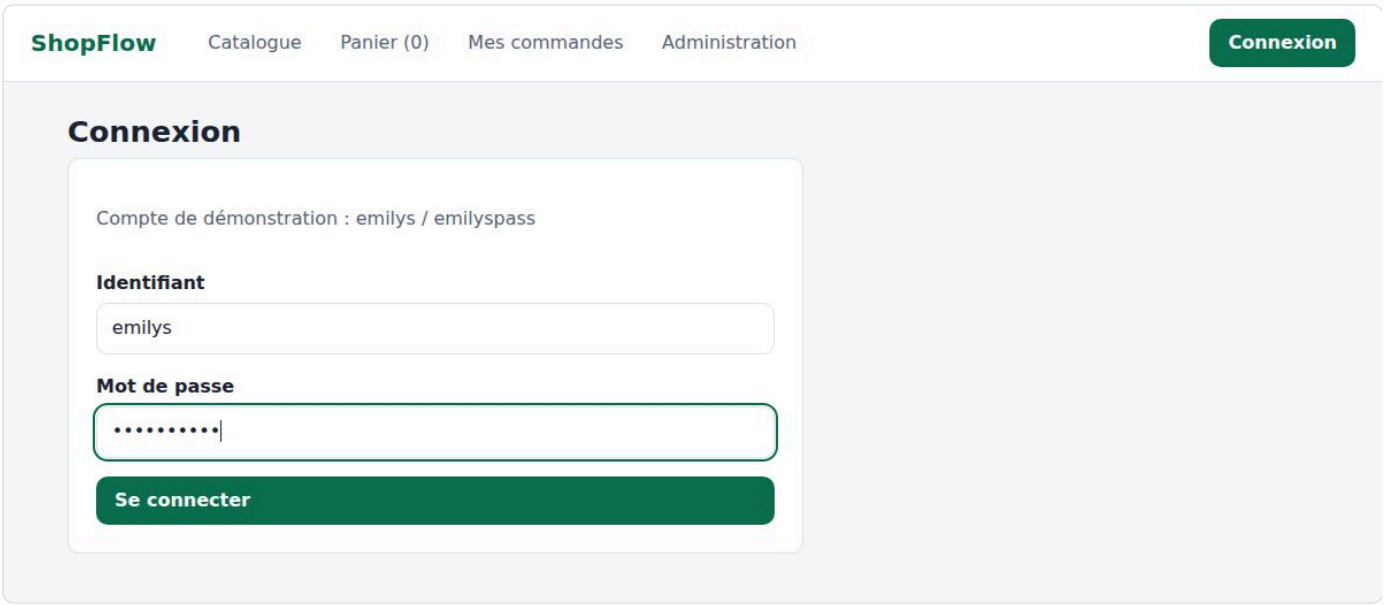
Demandez à un apprenant de valider le formulaire **entièrement vide** : la commande part quand même. Regardez l'adresse de la page de confirmation : le numéro et le montant sont dans l'URL (`?numero=209&total=…`), donc modifiables par n'importe qui.

Mes commandes, sans être connecté



/mes-commandes sans connexion : la page s'ouvre, l'API refuse (erreur 401 dans l'onglet Réseau), un message s'affiche.

La connexion



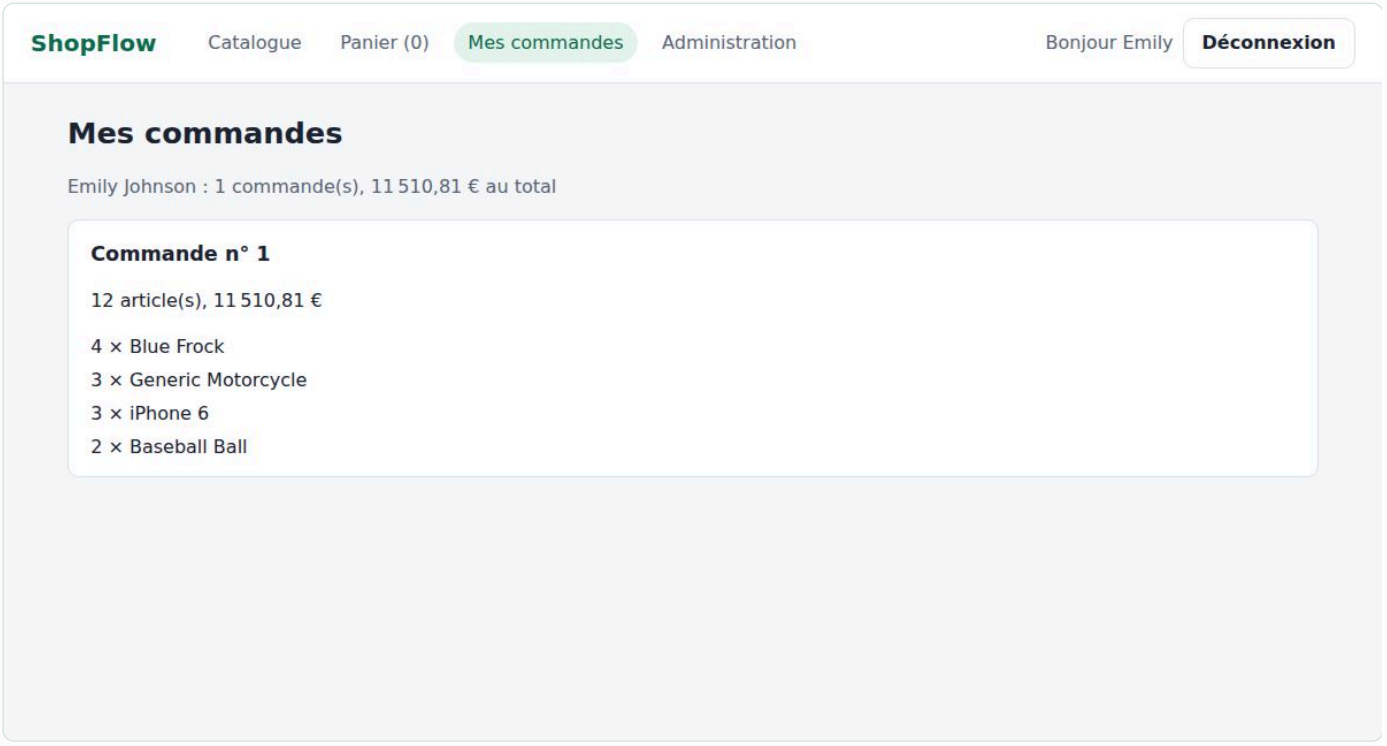
/connexion avec le compte de démonstration.



Une fois connecté, l'en-tête affiche « Bonjour Emily » et le bouton Déconnexion.

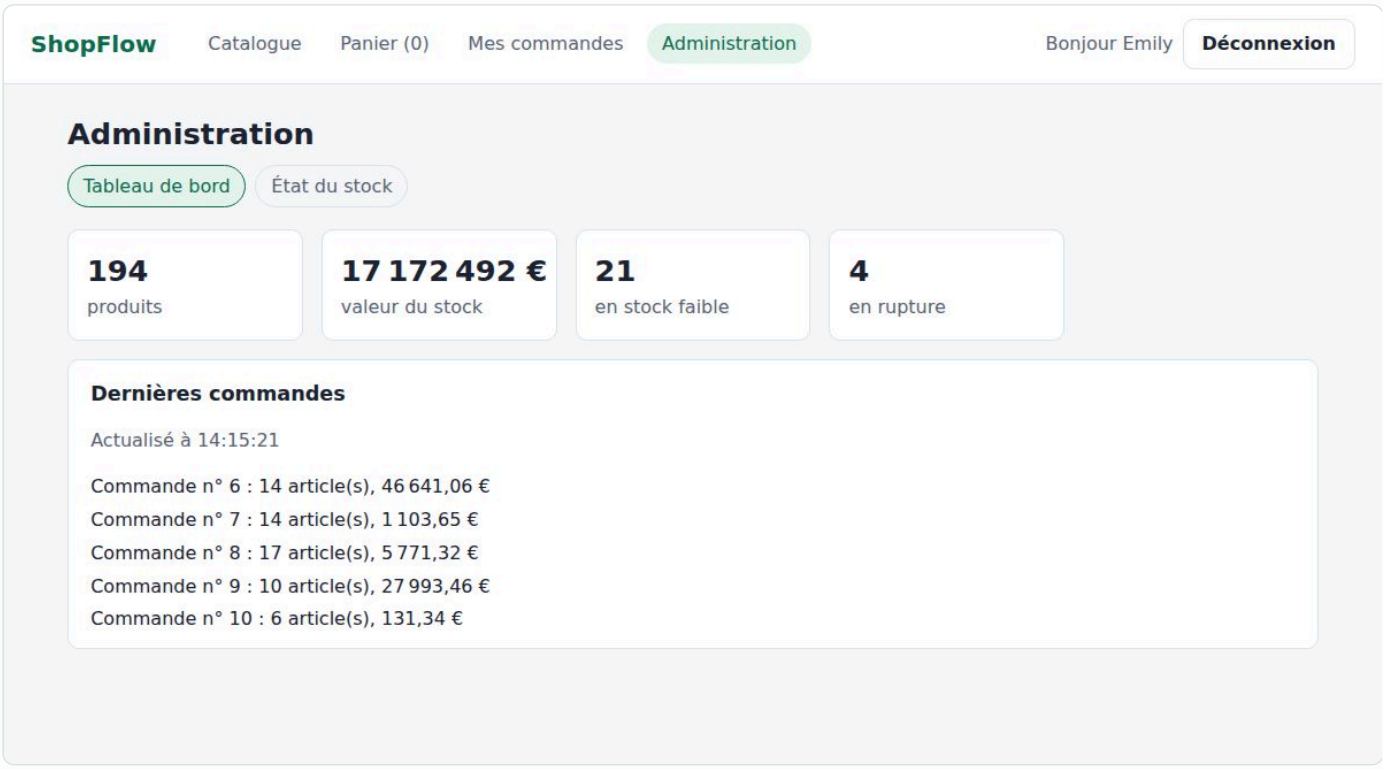
À montrer : Local Storage contient maintenant `shopflow-session`, avec le jeton `accessToken` en clair. Dans l'onglet Réseau, les requêtes suivantes portent un en-tête `Authorization: Bearer …`.

Mes commandes, connecté

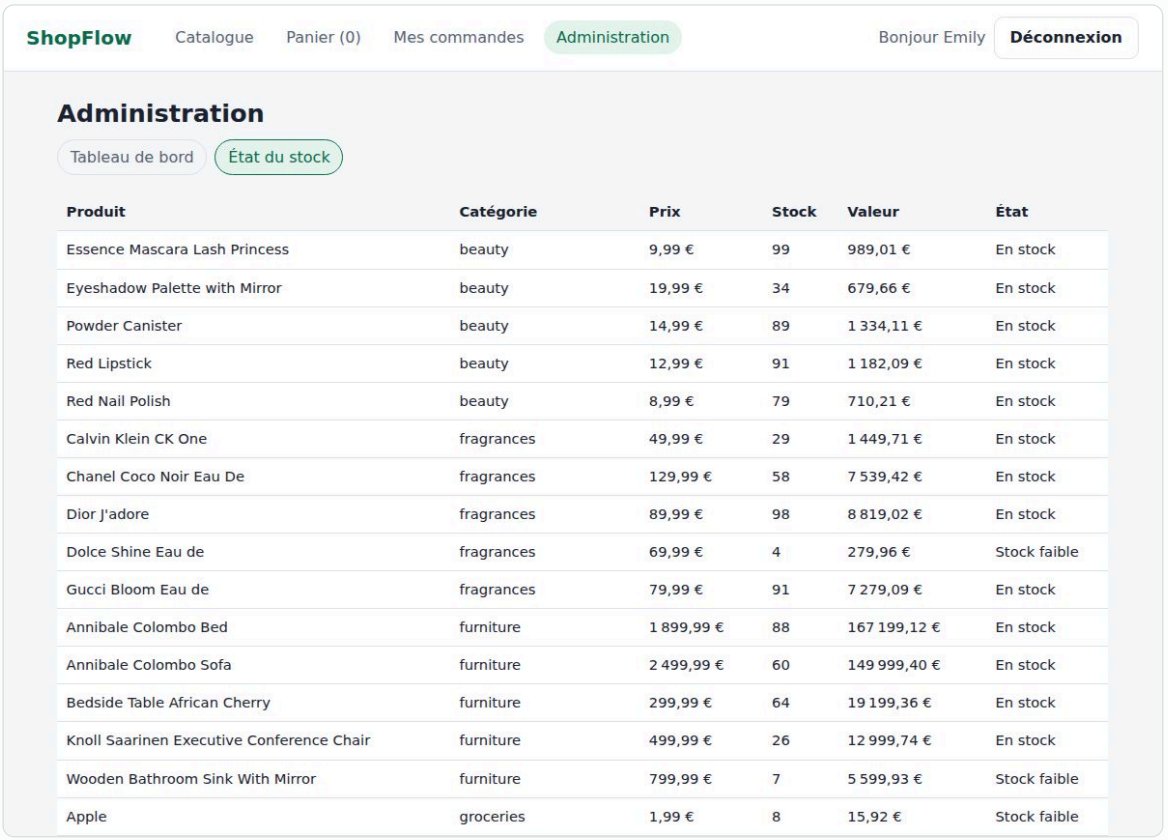


/mes-commandes : les commandes d'Emily (une commande de 12 articles), avec le total dépensé.

L'administration



/admin : indicateurs de stock, « Dernières commandes » actualisées toutes les 5 secondes.



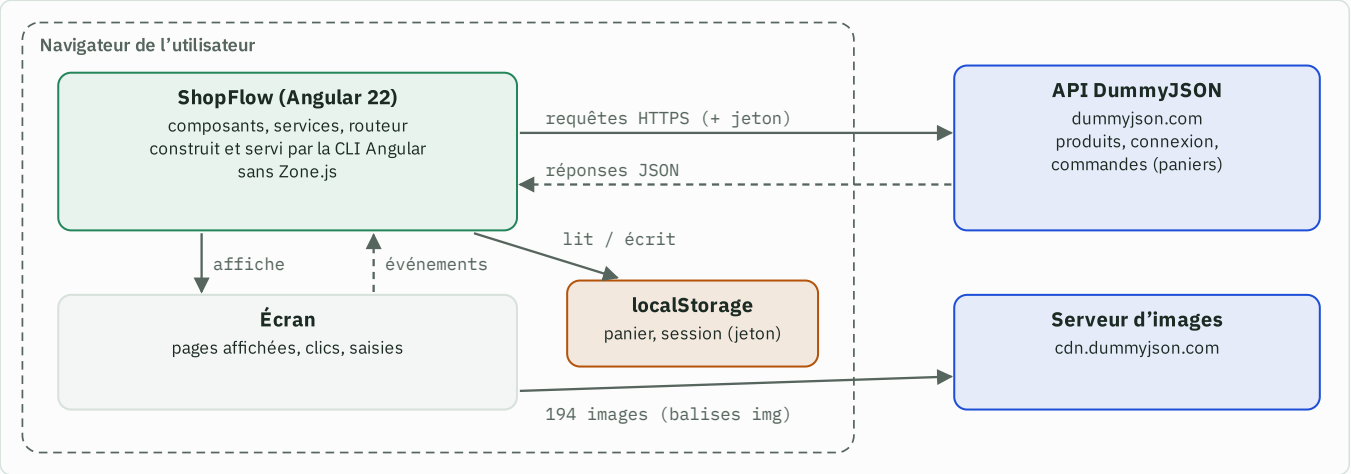
Produit	Catégorie	Prix	Stock	Valeur	État
Essence Mascara Lash Princess	beauty	9,99 €	99	989,01 €	En stock
Eyeshadow Palette with Mirror	beauty	19,99 €	34	679,66 €	En stock
Powder Canister	beauty	14,99 €	89	1 334,11 €	En stock
Red Lipstick	beauty	12,99 €	91	1 182,09 €	En stock
Red Nail Polish	beauty	8,99 €	79	710,21 €	En stock
Calvin Klein CK One	fragrances	49,99 €	29	1 449,71 €	En stock
Chanel Coco Noir Eau De	fragrances	129,99 €	58	7 539,42 €	En stock
Dior J'adore	fragrances	89,99 €	98	8 819,02 €	En stock
Dolce Shine Eau de	fragrances	69,99 €	4	279,96 €	Stock faible
Gucci Bloom Eau de	fragrances	79,99 €	91	7 279,09 €	En stock
Annibale Colombo Bed	furniture	1 899,99 €	88	167 199,12 €	En stock
Annibale Colombo Sofa	furniture	2 499,99 €	60	149 999,40 €	En stock
Bedside Table African Cherry	furniture	299,99 €	64	19 199,36 €	En stock
Knoll Saarinen Executive Conference Chair	furniture	499,99 €	26	12 999,74 €	En stock
Wooden Bathroom Sink With Mirror	furniture	799,99 €	7	5 599,93 €	Stock faible
Apple	groceries	1,99 €	8	15,92 €	Stock faible

/admin/stock : le tableau de l'état du stock, 194 lignes.

Déconnectez-vous puis ouvrez /admin : la page s'affiche quand même. N'importe quel visiteur voit le stock et les commandes. Dans l'onglet Réseau, une requête `carts?limit=5…` part toutes les 5 secondes, et **continue** après être revenu au catalogue.

3. L’organisation du code

Vue d’ensemble



ShopFlow s’exécute entièrement dans le navigateur. Il parle à l’API DummyJSON en HTTPS, garde le panier et la session dans le localStorage, et le navigateur télécharge les images directement sur le serveur d’images.

Le projet et ses dossiers

Terminal : shopflow (projet Angular CLI, fichiers de configuration secondaires masqués)

```
shopflow/
├── angular.json
├── package.json
├── public/
├── src/
│   ├── app/
│   │   ├── admin/
│   │   │   ├── admin-dashboard.html
│   │   │   ├── admin-dashboard.ts
│   │   │   ├── admin-layout.html
│   │   │   ├── admin-layout.ts
│   │   │   ├── live-orders.html
│   │   │   ├── live-orders.ts
│   │   │   ├── stock-table.html
│   │   │   └── stock-table.ts
│   │   ├── app.config.ts
│   │   ├── app.html
│   │   ├── app.routes.ts
│   │   ├── app.ts
│   │   ├── layout/
│   │   │   └── header/
│   │   │       ├── header.html
│   │   │       └── header.ts
│   │   ├── models/
│   │   │   ├── cart.ts
│   │   │   ├── order.ts
│   │   │   ├── product.ts
│   │   │   └── user.ts
│   │   ├── pages/
│   │   │   ├── cart/
│   │   │   │   ├── cart.html
│   │   │   │   └── cart.ts
│   │   │   ├── catalogue/
│   │   │   │   ├── catalogue.html
│   │   │   │   ├── catalogue.ts
│   │   │   │   ├── product-card.html
│   │   │   │   └── product-card.ts
│   │   │   ├── checkout/
│   │   │   │   ├── checkout.html
│   │   │   │   ├── checkout.ts
│   │   │   │   ├── confirmation.html
│   │   │   │   └── confirmation.ts
│   │   │   ├── login/
│   │   │   │   ├── login.html
│   │   │   │   └── login.ts
│   │   │   ├── orders/
│   │   │   │   ├── orders.html
│   │   │   │   └── orders.ts
│   │   │   └── product-detail/
│   │   │       ├── product-detail.html
│   │   │       └── product-detail.ts
│   │   └── services/
│   │       ├── api.ts
│   │       ├── auth-interceptor.ts
│   │       ├── auth-service.ts
│   │       ├── cart-service.ts
│   │       ├── order-service.ts
│   │       └── product-service.ts
│   ├── index.html
│   ├── main.ts
│   └── styles.css
└── tsconfig.json
```

Arborescence du projet : une seule application, tout le code dans src/app/.

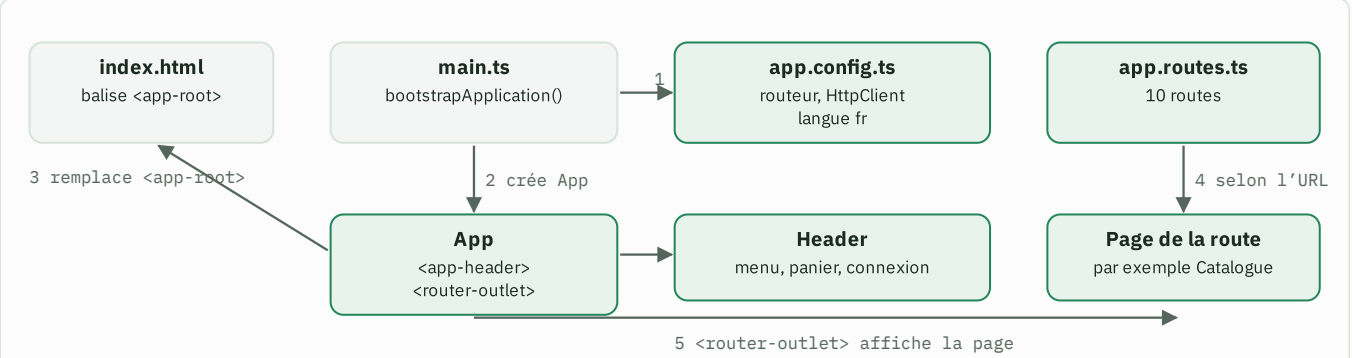
C’est la structure créée par `ng new` : `angular.json` décrit comment construire, servir et tester l’application, `src/` contient le code. Tout est dans une seule application, sans bibliothèque partagée ; c’est un des constats qui mèneront à installer Nx au module 1.

Dans `src/app/`, chaque dossier a un rôle :

- `models/` : les interfaces TypeScript qui décrivent les données de l’API (produit, ligne de panier, utilisateur, commande). Aucun code exécuté, seulement des types.

- `services/` : tout ce qui parle à l'API ou garde un état partagé (panier, session). Un composant ne fait jamais de requête lui-même.
- `layout/` : ce qui encadre toutes les pages ; ici l'en-tête.
- `pages/` : un dossier par page de l'application, avec ses composants propres (la carte produit est dans `pages/catalogue/`).
- `admin/` : l'espace d'administration, une mise en page avec un sous-menu et deux pages enfants (tableau de bord, état du stock).

Le démarrage



Du chargement de index.html à l'affichage de la première page.

src/main.ts

```ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

src/app/app.config.ts

```ts
import { ApplicationConfig, LOCALE_ID, provideBrowserGlobalErrorListeners } from '@angular/core';
import { registerLocaleData } from '@angular/common';
import localeFr from '@angular/common/locales/fr';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { authInterceptor } from './services/auth-interceptor';

registerLocaleData(localeFr);

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    { provide: LOCALE_ID, useValue: 'fr' },
  ],
};
```

src/app/app.ts `import { Component } from '@angular/core';`

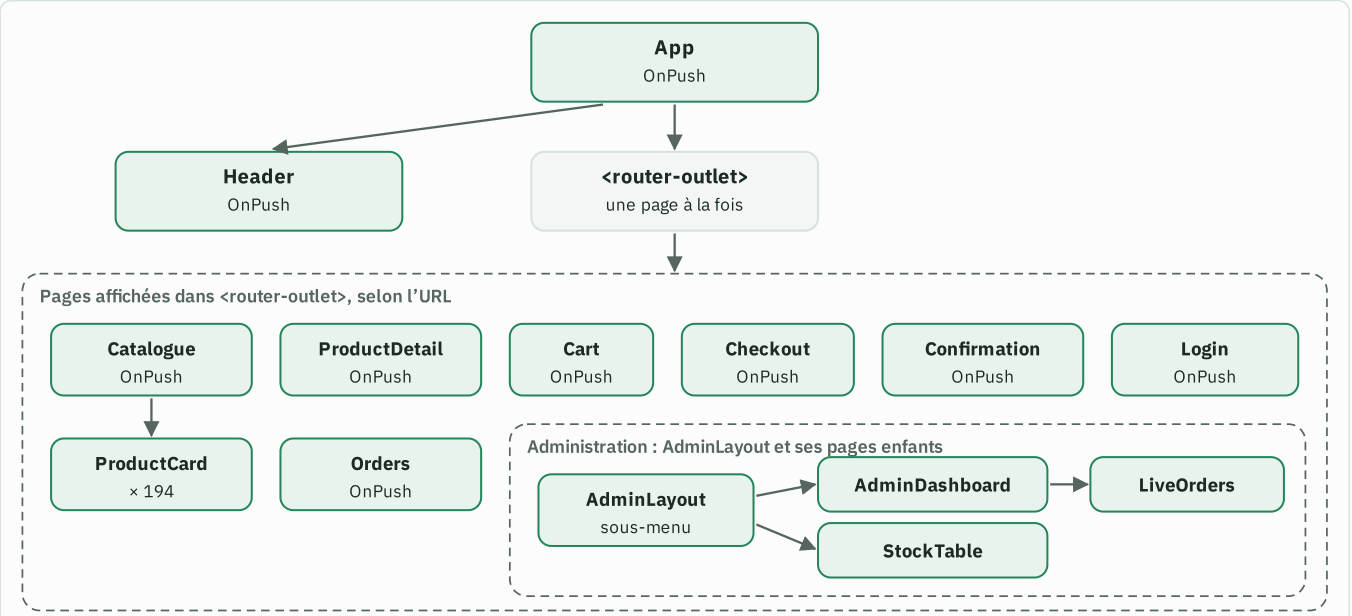
```ts
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { Header } from './layout/header/header';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Header],
  templateUrl: './app.html',
})
export class App {}
```

Dans le code existant

Avant les composants standalone, `main.ts` appelait `platformBrowserDynamic().bootstrapModule(AppModule)` et la configuration était répartie dans les `imports` et `providers` d'un NgModule racine ; les applications utilisaient aussi Zone.js pour savoir quand redessiner. Ce sont des points du programme qu'on explique sans les reproduire dans ShopFlow (modules 2 et 4).

L'arbre des composants



Tous les composants sont standalone et en OnPush, la valeur par défaut d'Angular 22 : un composant n'est redessiné que si un événement, une entrée ou le pipe async le signale.

À montrer : l'extension Angular DevTools (onglet Components) affiche ce même arbre dans le navigateur, avec les entrées de chaque composant.

Les routes

URL	Composant	Chargement	Protection
`/`	redirection vers /catalogue		
`/catalogue`	Catalogue	au démarrage	aucune
`/catalogue/:id`	ProductDetail	au démarrage	aucune
`/panier`	Cart	au démarrage	aucune
`/commande`	Checkout	au démarrage	aucune
`/commande/confirmation`	Confirmation	au démarrage	aucune
`/connexion`	Login	au démarrage	aucune
`/mes-commandes`	Orders	au démarrage	aucune (l'API refuse)
`/admin`	AdminLayout → AdminDashboard	au démarrage	aucune
`/admin/stock`	AdminLayout → StockTable	au démarrage	aucune
toute autre adresse	redirection vers /catalogue		pas de page 404

src/app/app.routes.ts — `import { Routes } from '@angular/router';`

```ts
import { Routes } from '@angular/router';
import { Catalogue } from './pages/catalogue/catalogue';
import { ProductDetail } from './pages/product-detail/product-detail';
import { Cart } from './pages/cart/cart';
import { Checkout } from './pages/checkout/checkout';
import { Confirmation } from './pages/checkout/confirmation';
import { Login } from './pages/login/login';
import { Orders } from './pages/orders/orders';
import { AdminLayout } from './admin/admin-layout';
import { AdminDashboard } from './admin/admin-dashboard';
import { StockTable } from './admin/stock-table';

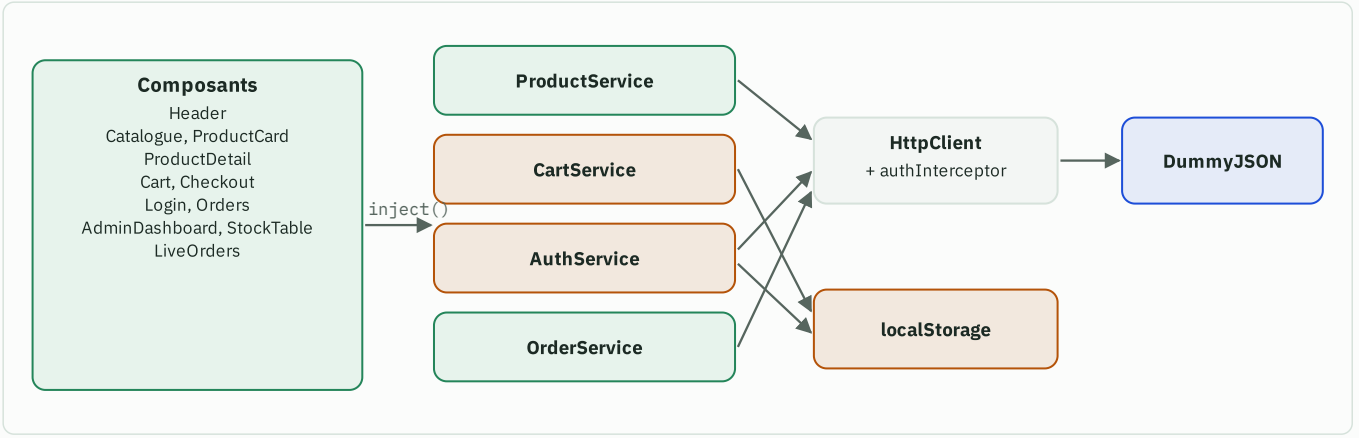
export const routes: Routes = [
  { path: '', redirectTo: 'catalogue', pathMatch: 'full' },
  { path: 'catalogue', component: Catalogue },
  { path: 'catalogue/:id', component: ProductDetail },
  { path: 'panier', component: Cart },
  { path: 'commande', component: Checkout },
  { path: 'commande/confirmation', component: Confirmation },
  { path: 'connexion', component: Login },
  { path: 'mes-commandes', component: Orders },
  {
    path: 'admin',
    component: AdminLayout,
    children: [
      { path: '', component: AdminDashboard },
      { path: 'stock', component: StockTable },
    ],
  },
  { path: '**', redirectTo: 'catalogue' },
];
```

Terminal : shopflow

```
$ npx ng build
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-VLKSZHRB.js    | main          | 340.28 kB |               90.81 kB
styles-SZZ3P6BQ.css | styles        |   6.36 kB |                1.60 kB
                    | Initial total | 346.63 kB |               92.41 kB
Application bundle generation complete. [6.428 seconds] - 2026-10-15T14:13:56.453Z
```

npx ng build : toutes les pages, administration comprise, sont dans un seul fichier main (340 ko), téléchargé avant le premier affichage.

Les services et qui les utilise



Trois couches : les composants demandent les services avec inject() ; les services font les requêtes avec HttpClient (l'intercepteur ajoute le jeton) ; CartService et AuthService gardent aussi leur état dans le localStorage.

Composant	ProductService	CartService	AuthService	OrderService	Router / ActivatedRoute
Header		count$	user$, logout()		Router
Catalogue	getAll(), getCategories()				
ProductCard		add()			
ProductDetail	getById()	add()			ActivatedRoute
Cart		lines$, total$, setQuantity(), remove(), clear()			
Checkout		total$, snapshot(), clear()	userId	create()	Router
Login			login()		Router
Orders			me()	byUser()	
AdminDashboard	getAll()				
StockTable	getAll()				
LiveOrders				latest()	

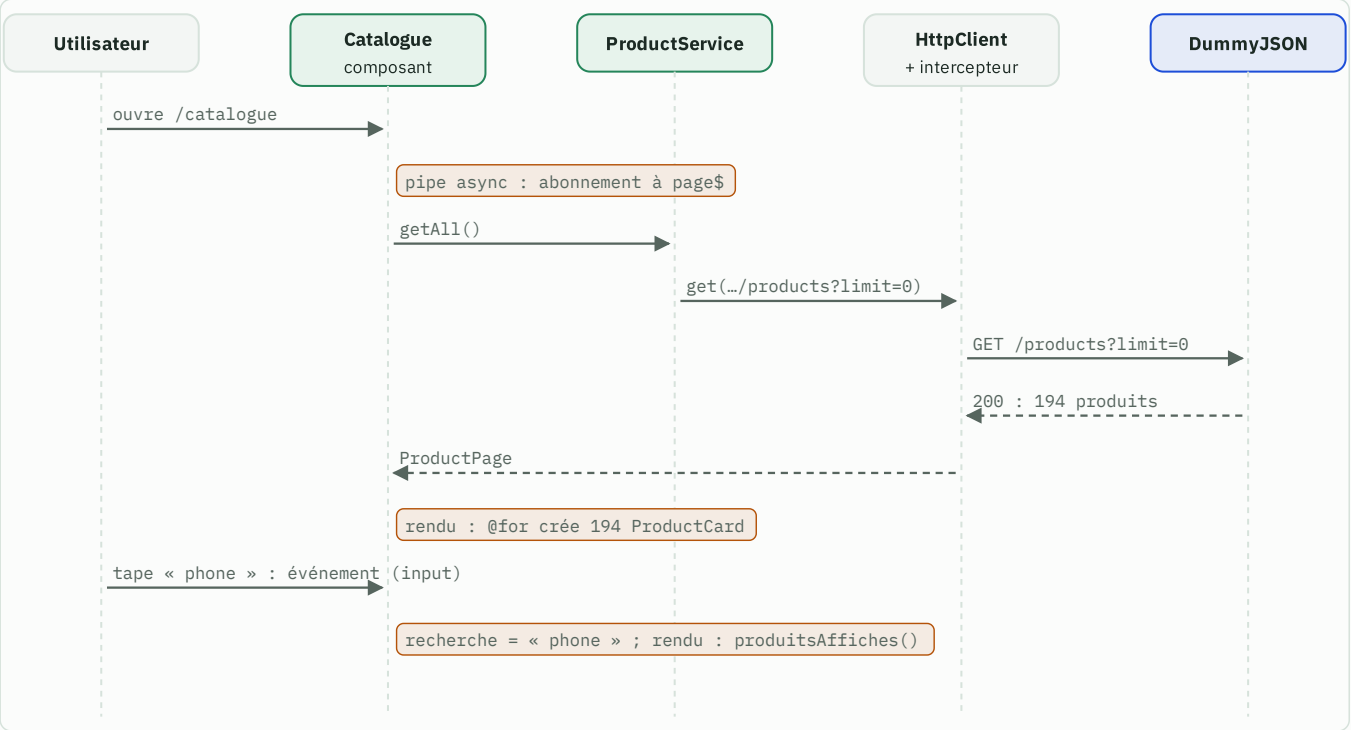
Tous les services sont décorés avec `@Service()` (Angular 22) : une seule instance pour toute l'application, créée au premier `inject()`. C'est ce qui permet à l'en-tête et à la page panier de voir le même panier.

> **Dans le code existant**
>
> Avant Angular 22, on écrivait `@Injectable({ providedIn: 'root' })` ; et avant Angular 14, les dépendances arrivaient par le constructeur : `constructor(private productService: ProductService) {}`.

4. Six interactions expliquées

Pour chaque interaction : le schéma au tableau, puis le code qui la réalise. Faites lire le schéma de gauche à droite et de haut en bas, une flèche à la fois.

Interaction 1 : afficher et filtrer le catalogue



Le pipe async déclenche la requête et réaffiche la page quand la réponse arrive. La recherche ne refait pas de requête : elle filtre la liste déjà reçue.

src/app/pages/catalogue/catalogue.ts — import { Component, inject } from

```ts
import { Component, inject } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { Product } from '../../models/product';
import { ProductService } from '../../services/product-service';
import { ProductCard } from './product-card';

type Tri = 'nom' | 'prix-croissant' | 'prix-decroissant';

@Component({
  selector: 'app-catalogue',
  imports: [AsyncPipe, ProductCard],
  templateUrl: './catalogue.html',
})
export class Catalogue {
  private readonly productService = inject(ProductService);

  protected readonly page$ = this.productService.getAll();
  protected readonly categories$ = this.productService.getCategories();
  protected recherche = '';
  protected categorie = '';
  protected tri: Tri = 'nom';

  protected produitsAffiches(produits: Product[]): Product[] {
```

```
    const terme = this.recherche.trim().toLowerCase();
    const filtres = produits.filter(
      (p) => p.title.toLowerCase().includes(terme) && (!this.categorie || p.category === this.categorie),
    );
    return [...filtres].sort((a, b) =>
      this.tri === 'nom' ? a.title.localeCompare(b.title) : this.tri === 'prix-croissant' ? a.price - b.price
: b.price - a.price,
    );
  }

  protected onRecherche(event: Event): void {
    this.recherche = (event.target as HTMLInputElement).value;
  }

  protected onCategorie(event: Event): void {
    this.categorie = (event.target as HTMLSelectElement).value;
  }

  protected onTri(event: Event): void {
    this.tri = (event.target as HTMLSelectElement).value as Tri;
  }
}
```

src/app/pages/catalogue/catalogue.html <h1>Catalogue</h1>

```
<h1>Catalogue</h1>
@if (page$ | async; as page) {
  <p class="sous-titre">{{ page.total }} produits</p>
  <div class="barre-outils">
    <input class="champ recherche" type="search" placeholder="Rechercher un produit…"
(input)="onRecherche($event)" />
    <select class="champ" (change)="onCategorie($event)">
      <option value="">Toutes les catégories</option>
      @for (c of categories$ | async; track c.slug) {
        <option [value]="c.slug">{{ c.name }}</option>
      }
    </select>
    <select class="champ" (change)="onTri($event)">
      <option value="nom">Trier par nom</option>
      <option value="prix-croissant">Prix croissant</option>
      <option value="prix-decroissant">Prix décroissant</option>
    </select>
  </div>
  <div class="grille">
    @for (p of produitsAffiches(page.products); track $index) {
      <app-product-card [product]="p" />
    } @empty {
      <p class="info">Aucun produit ne correspond à votre recherche.</p>
    }
  </div>
} @else {
  <p class="info">Chargement du catalogue…</p>
}
```

src/app/pages/catalogue/product-card.ts

```
import { Component, inject, input } from '@angular/core';
import { CurrencyPipe, DecimalPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Product, prixRemise } from '../../models/product';
import { CartService } from '../../services/cart-service';

@Component({
  selector: 'app-product-card',
  imports: [CurrencyPipe, DecimalPipe, RouterLink],
  templateUrl: './product-card.html',
})
export class ProductCard {
  readonly product = input.required<Product>();
  private readonly cart = inject(CartService);

  protected prixFinal(): number {
    return prixRemise(this.product());
  }

  protected ajouter(): void {
    this.cart.add(this.product());
  }
}
```

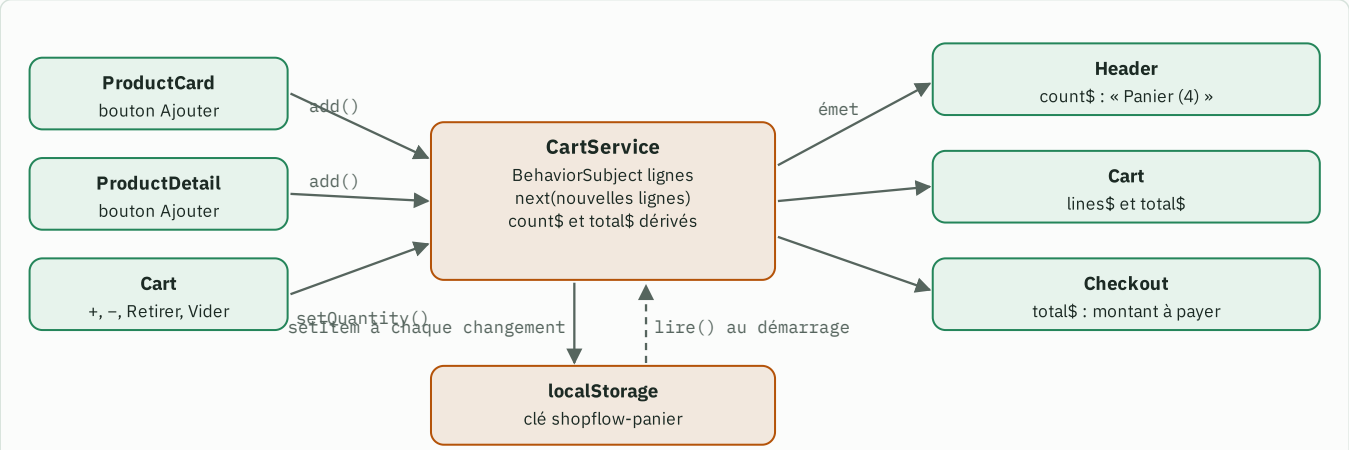
src/app/pages/catalogue/product-card.html

```
@let p = product();
<article class="carte">
  <a [routerLink]="['/catalogue', p.id]">
    <img class="carte-image" [src]="p.thumbnail" [alt]="p.title" width="300" height="300" />
  </a>
  <div class="carte-corps">
    <p class="carte-categorie">{{ p.category }}</p>
    <h2 class="carte-titre"><a [routerLink]="['/catalogue', p.id]">{{ p.title }}</a></h2>
    <p class="carte-note">★ {{ p.rating | number: '1.1-1' }}</p>
    <p class="carte-prix">
      {{ prixFinal() | currency: 'EUR' }}
      @if (p.discountPercentage > 0) {
        <s class="prix-barre">{{ p.price | currency: 'EUR' }}</s>
      }
    </p>
    <button class="bouton" type="button" (click)="ajouter()">Ajouter au panier</button>
  </div>
</article>
```

Ce qu'on remarque

`produitsAffiches()` filtre et trie 194 produits **à chaque rendu**, et `prixFinal()` est appelée par chaque carte à chaque rendu. `track $index` associe les cartes à des positions : un filtre redessine les cartes au lieu de les déplacer. Les 194 images partent dès l'ouverture. Tout cela se mesure au module 1.

Interaction 2 : ajouter au panier



Un seul objet garde le panier : les composants l'appellent pour le modifier, il émet la nouvelle liste à tous ses abonnés et l'écrit dans le localStorage.

Le `BehaviorSubject` est un Observable qui **garde la dernière valeur** : un composant qui s'abonne la reçoit tout de suite, puis reçoit chaque nouvelle valeur. Chaque méthode crée un **nouveau tableau** au lieu de modifier l'ancien ; on verra au module 3 pourquoi c'est indispensable.

src/app/services/cart-service.ts

```
import { Service } from '@angular/core';
import { BehaviorSubject, map } from 'rxjs';
import { CartLine } from '../models/cart';
import { Product, prixRemise } from '../models/product';

const CLE_STOCKAGE = 'shopflow-panier';

@Service()
export class CartService {
  private readonly lignes = new BehaviorSubject<CartLine[]>(this.lire());
  readonly lines$ = this.lignes.asObservable();
  readonly count$ = this.lines$.pipe(map((lignes) => lignes.reduce((n, l) => n + l.quantity, 0)));
  readonly total$ = this.lines$.pipe(map((lignes) => lignes.reduce((s, l) => s + l.price * l.quantity, 0)));

  add(product: Product): void {
    const lignes = this.lignes.value;
    const existante = lignes.find((l) => l.productId === product.id);
    const suivantes = existante
      ? lignes.map((l) => (l.productId === product.id ? { ...l, quantity: l.quantity + 1 } : l))
      : [...lignes, { productId: product.id, title: product.title, price: prixRemise(product), quantity: 1,
thumbnail: product.thumbnail }];
    this.publier(suivantes);
  }
```

```
  setQuantity(productId: number, quantity: number): void {
    const suivantes =
      quantity <= 0
        ? this.lignes.value.filter((l) => l.productId !== productId)
        : this.lignes.value.map((l) => (l.productId === productId ? { ...l, quantity } : l));
    this.publier(suivantes);
  }

  remove(productId: number): void {
    this.publier(this.lignes.value.filter((l) => l.productId !== productId));
  }

  clear(): void {
    this.publier([]);
  }

  snapshot(): CartLine[] {
    return this.lignes.value;
  }

  private publier(lignes: CartLine[]): void {
    this.lignes.next(lignes);
    localStorage.setItem(CLE_STOCKAGE, JSON.stringify(lignes));
  }

  private lire(): CartLine[] {
    const brut = localStorage.getItem(CLE_STOCKAGE);
    return brut ? JSON.parse(brut) : [];
  }
}
```

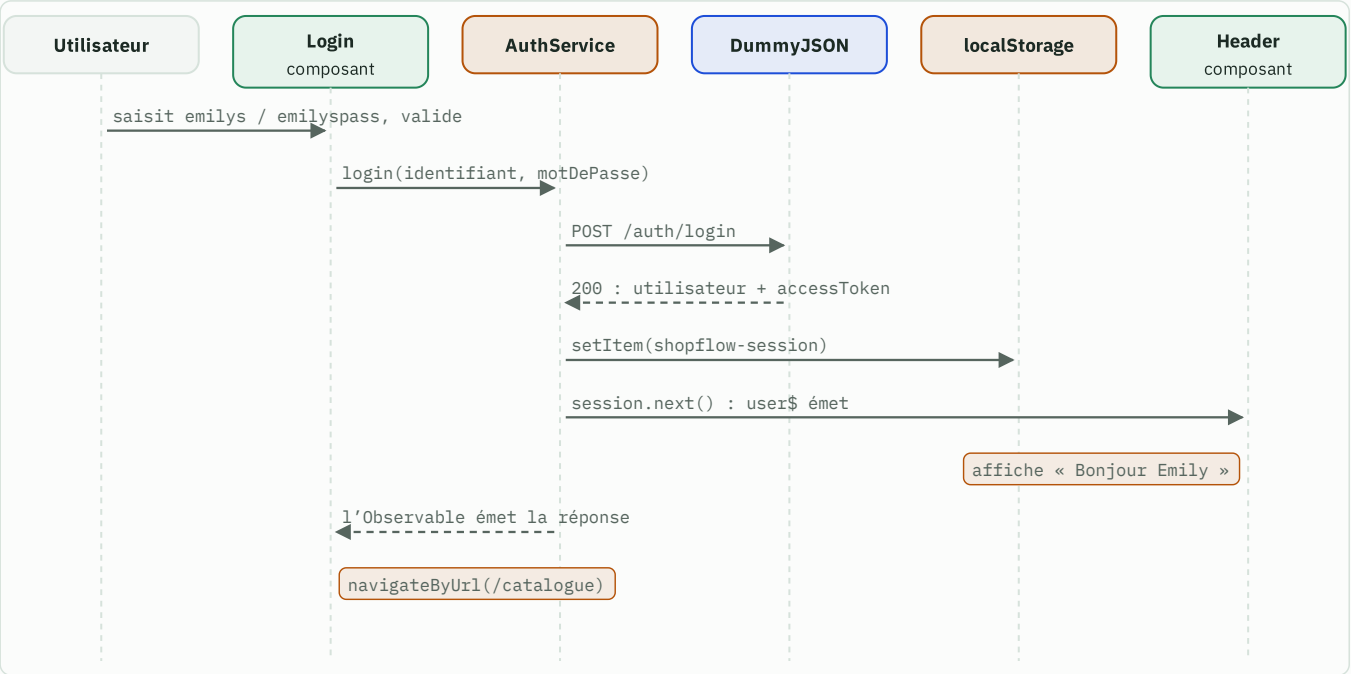
src/app/layout/header/header.html — `<header class="entete">`

```
<header class="entete">
  <a class="logo" routerLink="/">ShopFlow</a>
  <nav class="nav">
    <a routerLink="/catalogue" routerLinkActive="actif">Catalogue</a>
    <a routerLink="/panier" routerLinkActive="actif">Panier ({{ nbArticles$ | async }})</a>
    <a routerLink="/mes-commandes" routerLinkActive="actif">Mes commandes</a>
    <a routerLink="/admin" routerLinkActive="actif">Administration</a>
  </nav>
  <div class="entete-actions">
    @if (utilisateur$ | async; as u) {
      <span class="info">Bonjour {{ u.firstName }}</span>
      <button class="bouton bouton-secondaire" type="button" (click)="deconnecter()">Déconnexion</button>
    } @else {
      <a class="bouton" routerLink="/connexion">Connexion</a>
    }
  </div>
</header>
```

Ce qu'on remarque

L'état du panier repose sur RxJS (`BehaviorSubject` , `map`) : c'est le composant que le TP du module 3 réécrira avec des signaux. La lecture du localStorage n'est pas protégée : un contenu invalide fait planter l'application au démarrage (faites-le en direct en modifiant la clé dans l'onglet Application).

Interaction 3 : se connecter



La connexion : le service enregistre la session et prévient l'en-tête ; le composant ne fait que naviguer.

src/app/services/auth-service.ts — import { HttpClient } from

```ts
import { HttpClient } from '@angular/common/http';
import { inject, Service } from '@angular/core';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { LoginResponse, User } from '../models/user';
import { API_URL } from './api';

const CLE_SESSION = 'shopflow-session';

@Service()
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly session = new BehaviorSubject<LoginResponse | null>(this.lire());
  readonly user$ = this.session.asObservable();

  get token(): string | null {
    return this.session.value?.accessToken ?? null;
  }

  get userId(): number | null {
    return this.session.value?.id ?? null;
  }

  login(username: string, password: string): Observable<LoginResponse> {
```

```
    return this.http
      .post<LoginResponse>(`${API_URL}/auth/login`, { username, password, expiresInMins: 60 })
      .pipe(
        tap((reponse) => {
          localStorage.setItem(CLE_SESSION, JSON.stringify(reponse));
          this.session.next(reponse);
        }),
      );
  }

  me(): Observable<User> {
    return this.http.get<User>(`${API_URL}/auth/me`);
  }

  logout(): void {
    localStorage.removeItem(CLE_SESSION);
    this.session.next(null);
  }

  private lire(): LoginResponse | null {
    const brut = localStorage.getItem(CLE_SESSION);
    return brut ? JSON.parse(brut) : null;
  }
}
```

Ensuite, **chaque** requête de HttpClient passe par l'intercepteur, qui ajoute le jeton :

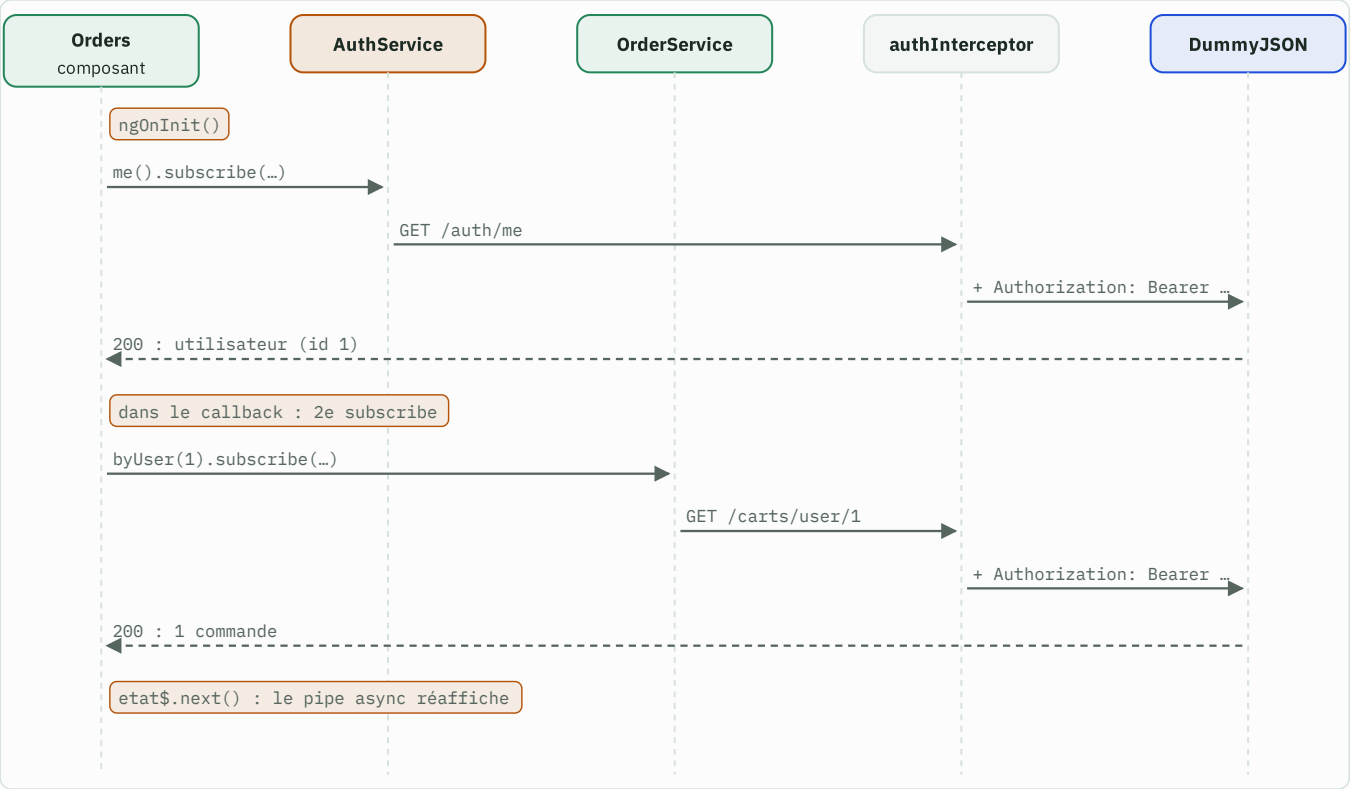
src/app/services/auth-interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from './auth-service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  if (!token) {
    return next(req);
  }
  return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
};
```

> **Ce qu'on remarque**
>
> Le jeton est dans le localStorage, lisible par n'importe quel script de la page. Après la connexion, l'utilisateur est toujours renvoyé au catalogue, même s'il venait de « Mes commandes ». Et rien ne protège les pages : c'est le travail des guards du module 2.

Interaction 4 : afficher mes commandes



Deux requêtes en cascade, la seconde lancée dans le callback de la première. Sans jeton, /auth/me répond 401 et la page affiche « Connectez-vous… ».

src/app/pages/orders/orders.ts — `import { Component, inject, OnInit } from`

```ts
import { Component, inject, OnInit } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { BehaviorSubject } from 'rxjs';
import { Order } from '../../models/order';
import { User } from '../../models/user';
import { AuthService } from '../../services/auth-service';
import { OrderService } from '../../services/order-service';

interface EtatCommandes {
  chargement: boolean;
  erreur?: string;
  utilisateur?: User;
  commandes: Order[];
  totalDepense: number;
}

@Component({
  selector: 'app-orders',
  imports: [AsyncPipe, CurrencyPipe],
  templateUrl: './orders.html',
})
export class Orders implements OnInit {
  private readonly auth = inject(AuthService);
  private readonly orderService = inject(OrderService);

  protected readonly etat$ = new BehaviorSubject<EtatCommandes>({ chargement: true, commandes: [],
```

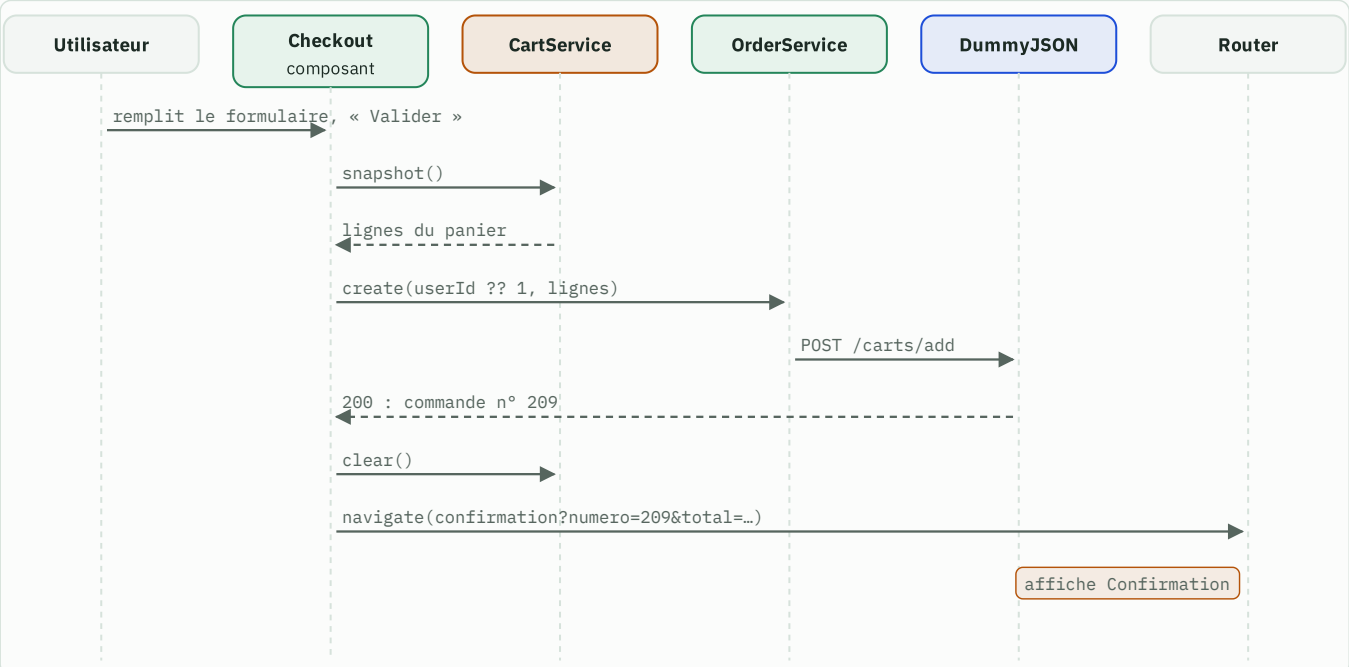
```
totalDepense: 0 });

  ngOnInit(): void {
    this.auth.me().subscribe({
      next: (utilisateur) => {
        this.orderService.byUser(utilisateur.id).subscribe({
          next: (page) =>
            this.etat$.next({
              chargement: false,
              utilisateur,
              commandes: page.carts,
              totalDepense: page.carts.reduce((s, c) => s + c.discountedTotal, 0),
            }),
          error: () =>
            this.etat$.next({ chargement: false, erreur: 'Impossible de charger vos commandes.', commandes:
[], totalDepense: 0 }),
        });
      },
      error: () =>
        this.etat$.next({ chargement: false, erreur: 'Connectez-vous pour voir vos commandes.', commandes: [],
totalDepense: 0 }),
    });
  }
}
```

Ce qu'on remarque

Des `subscribe` imbriqués, l'état complet réécrit dans trois branches, et aucune requête annulée si l'on quitte la page avant la réponse. Le module 2 réécrit ce composant en un seul flux RxJS.

Interaction 5 : passer commande



La commande : lecture du panier, envoi à l'API, panier vidé, puis navigation vers la confirmation avec le numéro dans l'URL.

src/app/pages/checkout/checkout.ts

```
import { Component, inject } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth-service';
import { CartService } from '../../services/cart-service';
import { OrderService } from '../../services/order-service';

@Component({
  selector: 'app-checkout',
  imports: [AsyncPipe, CurrencyPipe, FormsModule],
  templateUrl: './checkout.html',
})
export class Checkout {
  private readonly cart = inject(CartService);
  private readonly orderService = inject(OrderService);
  private readonly auth = inject(AuthService);
  private readonly router = inject(Router);

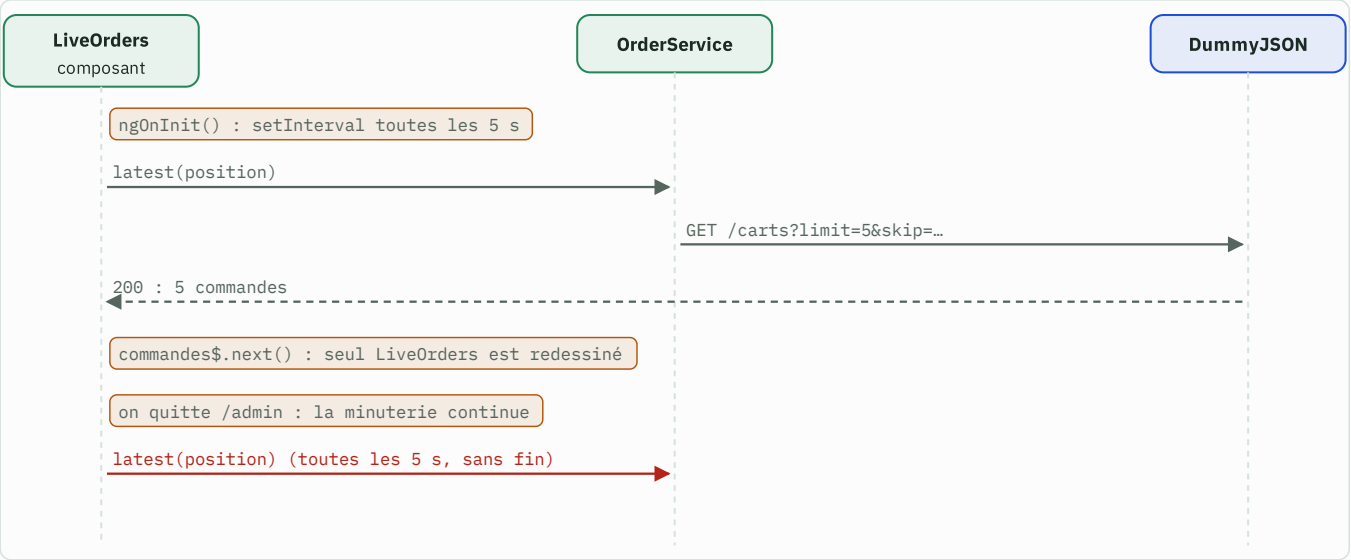
  protected readonly total$ = this.cart.total$;
  protected client = { nom: '', email: '', adresse: '', codePostal: '', ville: '' };

  protected valider(): void {
    this.orderService.create(this.auth.userId ?? 1, this.cart.snapshot()).subscribe((commande) => {
      this.cart.clear();
      this.router.navigate(['/commande/confirmation'], { queryParams: { numero: commande.id, total:
commande.total } });
    });
  }
}
```

Ce qu'on remarque

Aucune validation (champs vides, e-mail invalide, panier vide), une commande attribuée à l'utilisateur 1 si personne n'est connecté, aucun message en cas d'erreur de l'API, un double clic qui envoie deux commandes. Le module 6 refait ce formulaire avec FormBuilder et des validations.

Interaction 6 : le tableau de bord de l'administration



Le tableau de bord interroge l'API toutes les 5 secondes. Grâce à OnPush et au pipe async, seul le bloc des dernières commandes est redessiné ; mais rien n'arrête la minuterie.

src/app/admin/admin-layout.html `<h1>Administration</h1>`

```
<h1>Administration</h1>
<nav class="sous-nav">
  <a routerLink="/admin" routerLinkActive="actif" [routerLinkActiveOptions]="{ exact: true }">Tableau de bord</a>
  <a routerLink="/admin/stock" routerLinkActive="actif">État du stock</a>
</nav>
<router-outlet />
```

src/app/admin/admin-dashboard.ts `import { Component, inject } from`

```
import { Component, inject } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { Product } from '../models/product';
import { ProductService } from '../services/product-service';
import { LiveOrders } from './live-orders';

@Component({
  selector: 'app-admin-dashboard',
  imports: [AsyncPipe, CurrencyPipe, LiveOrders],
  templateUrl: './admin-dashboard.html',
})
export class AdminDashboard {
  protected readonly page$ = inject(ProductService).getAll();

  protected valeurStock(produits: Product[]): number {
    return produits.reduce((s, p) => s + p.price * p.stock, 0);
  }

  protected nbStockFaible(produits: Product[]): number {
    return produits.filter((p) => p.stock > 0 && p.stock < 10).length;
  }
```

```
  protected nbRuptures(produits: Product[]): number {
    return produits.filter((p) => p.stock === 0).length;
  }
}
```

src/app/admin/live-orders.ts

```
import { Component, inject, OnInit } from '@angular/core';
import { AsyncPipe, CurrencyPipe, DatePipe } from '@angular/common';
import { BehaviorSubject } from 'rxjs';
import { Order } from '../models/order';
import { OrderService } from '../services/order-service';

@Component({
  selector: 'app-live-orders',
  imports: [AsyncPipe, CurrencyPipe, DatePipe],
  templateUrl: './live-orders.html',
})
export class LiveOrders implements OnInit {
  private readonly orderService = inject(OrderService);
  protected readonly commandes$ = new BehaviorSubject<Order[]>([]);
  protected readonly miseAJour$ = new BehaviorSubject<Date | null>(null);
  private position = 0;

  ngOnInit(): void {
    this.charger();
    setInterval(() => this.charger(), 5000);
  }

  private charger(): void {
    // Simulation : l'API de démonstration ne reçoit pas de nouvelles commandes, on fait défiler ses paniers
    this.position = (this.position + 5) % 200;
    this.orderService.latest(this.position).subscribe((page) => {
      this.commandes$.next(page.carts);
      this.miseAJour$.next(new Date());
    });
  }
}
```

src/app/admin/stock-table.ts

```
import { Component, inject } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { Product } from '../models/product';
import { ProductService } from '../services/product-service';

@Component({
  selector: 'app-stock-table',
  imports: [AsyncPipe, CurrencyPipe],
  templateUrl: './stock-table.html',
})
export class StockTable {
  protected readonly page$ = inject(ProductService).getAll();
```

```
  protected etatStock(p: Product): string {
    return p.stock === 0 ? 'Rupture' : p.stock < 10 ? 'Stock faible' : 'En stock';
  }

  protected valeur(p: Product): number {
    return p.price * p.stock;
  }
}
```

Ce qu'on remarque

La minuterie n'est jamais arrêtée : après avoir quitté l'administration, les requêtes continuent toutes les 5 secondes, et chaque visite en ajoute une nouvelle. Le catalogue complet est chargé trois fois (catalogue, tableau de bord, stock). Le tableau du stock appelle deux méthodes pour chacune de ses 194 lignes. Le module 2 traite les deux premiers points avec RxJS, le TP du module 1 le troisième.

5. Ce qu'on a remarqué

Terminez la visite en complétant au tableau la liste des constats avec les apprenants. Elle sert de point de départ à la séquence suivante : le diagnostic outillé (mesures) puis la rédaction du cahier des charges.

Constat	Où on l'a vu	Module qui le traite
Tout le catalogue et ses 194 images chargés d'un coup	Onglet Réseau, catalogue	1 (chargement différé, optimisation)
Toutes les pages dans le paquet initial (340 ko)	Sortie de `ng build`	1 (component-level lazy loading)
Une seule application, sans outil de monorepo ni bibliothèque ; carte couplée au panier	Arborescence, ProductCard	1 (Nx, architecture)
Méthodes appelées dans les templates, `track $index`	Catalogue, carte, administration	1 (TP analyser et optimiser)
Aucune documentation du code	Le dépôt	1 (documentation automatisée)
Minuterie jamais arrêtée : les requêtes continuent après avoir quitté l'administration	Onglet Réseau	2 (RxJS)
Catalogue complet chargé trois fois (catalogue, tableau de bord, stock)	Onglet Réseau	2 (RxJS)
`subscribe` imbriqués, erreurs gérées à la main	Mes commandes	2 (RxJS)
Pages privées accessibles sans connexion ; retour forcé au catalogue	Administration, Mes commandes, Connexion	2 (guards, resolver)
État du panier et de la session en `BehaviorSubject`	CartService, AuthService	3 (signaux)
Textes écrits en dur en français	Tous les templates	5 (internationalisation)
Formulaire de commande sans validation, commande sans utilisateur	Commande	6 (formulaires dynamiques)
Prix de la fiche différent de celui de la carte ; aucun test	Fiche produit	7 (tests)

Constat	Où on l'a vu	Module qui le traite
Page vide pour un moteur de recherche (rendu dans le navigateur seulement)	Afficher le code source de la page (Ctrl+U)	8 (rendu côté serveur)

Pour le dernier point, faites afficher le code source de la page catalogue (Ctrl+U) : il ne contient que `<app-root></app-root>`. Un moteur de recherche ou un aperçu de lien sur un réseau social ne voit aucun produit.

10 h 45 – 11 h 00 : pause

`11 h 00 – 12 h 30, 13 h 30 – 14 h 30`

Séquence 2 : diagnostiquer ShopFlow

Au programme de cette séquence (2 h 30)

Point du programme : module 1, « TP : analyser et optimiser une application », partie analyse. L'optimisation se fait ensuite, module par module.

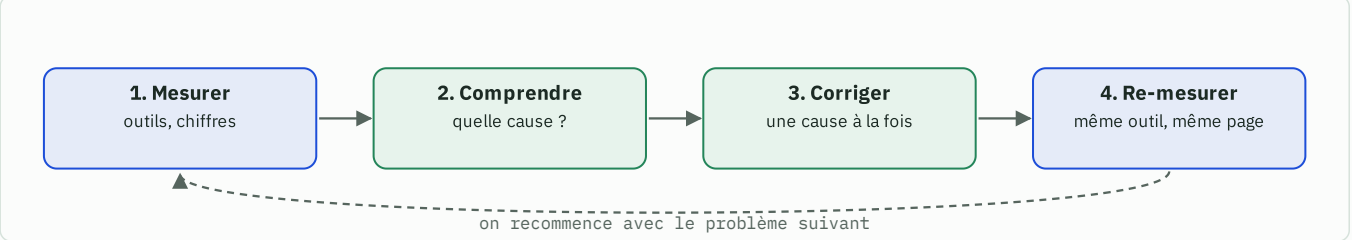
Objectif : à la fin de la séquence, les apprenants savent mesurer une application Angular avec cinq outils et ont rédigé ensemble le cahier des charges qui sert de fil conducteur aux trois jours.

- 15 min : pourquoi mesurer avant de corriger
- 75 min : les cinq outils, démontrés par le formateur sur ShopFlow
- 45 min : TP en binômes, compléter le diagnostic
- 15 min : mise en commun, cahier des charges

Avant la séquence : ShopFlow tourne (`npm start`), Chrome est ouvert avec l'extension Angular DevTools installée.

1. Pourquoi mesurer avant de corriger

En entreprise, on hérite rarement d'une application neuve : on reprend une application qui fonctionne, écrite par d'autres, et on doit l'améliorer sans rien casser. Avant de modifier une ligne, on mesure. La mesure dit ce qui pose vraiment problème, et la même mesure refaite après la correction prouve le gain.



La démarche suivie pendant toute la formation.

2. Outil 1 : Lighthouse, la performance perçue

Le problème : ShopFlow paraît lente sur un téléphone. Lighthouse, intégré à Chrome, simule un téléphone moyen sur un réseau lent et note la page sur 100.

Point important : on mesure toujours la version de production. `ng serve` produit une version de développement, non optimisée, dont les chiffres ne veulent rien dire.

```
Terminal 1 — dans shopflow/ (racine du projet, là où se trouve package.json) · construire puis servir la version de production
```

npx ng build

```
npx ng build
npx serve -s dist/shopflow/browser -l 8080
```

`-s` renvoie `index.html` pour toute adresse inconnue : Angular affiche alors la bonne page. Sans cette option, seule la racine fonctionne ; `/panier` ou `/catalogue/1` répondent 404, et Lighthouse échoue.

Ensuite, dans Chrome : ouvrir `http://localhost:8080`, touche F12, onglet **Lighthouse**, mode **Navigation**, appareil **Mobile**, catégorie **Performance**, bouton **Analyser le chargement de la page**. La même mesure existe en ligne de commande :

```
Terminal 2 — n'importe quel dossier (le serveur tourne dans le terminal 1) · le rapport s'ouvre dans le navigateur
```

npx lighthouse http://localhost:8080

```
npx lighthouse http://localhost:8080 --only-categories=performance --form-factor=mobile --view
```

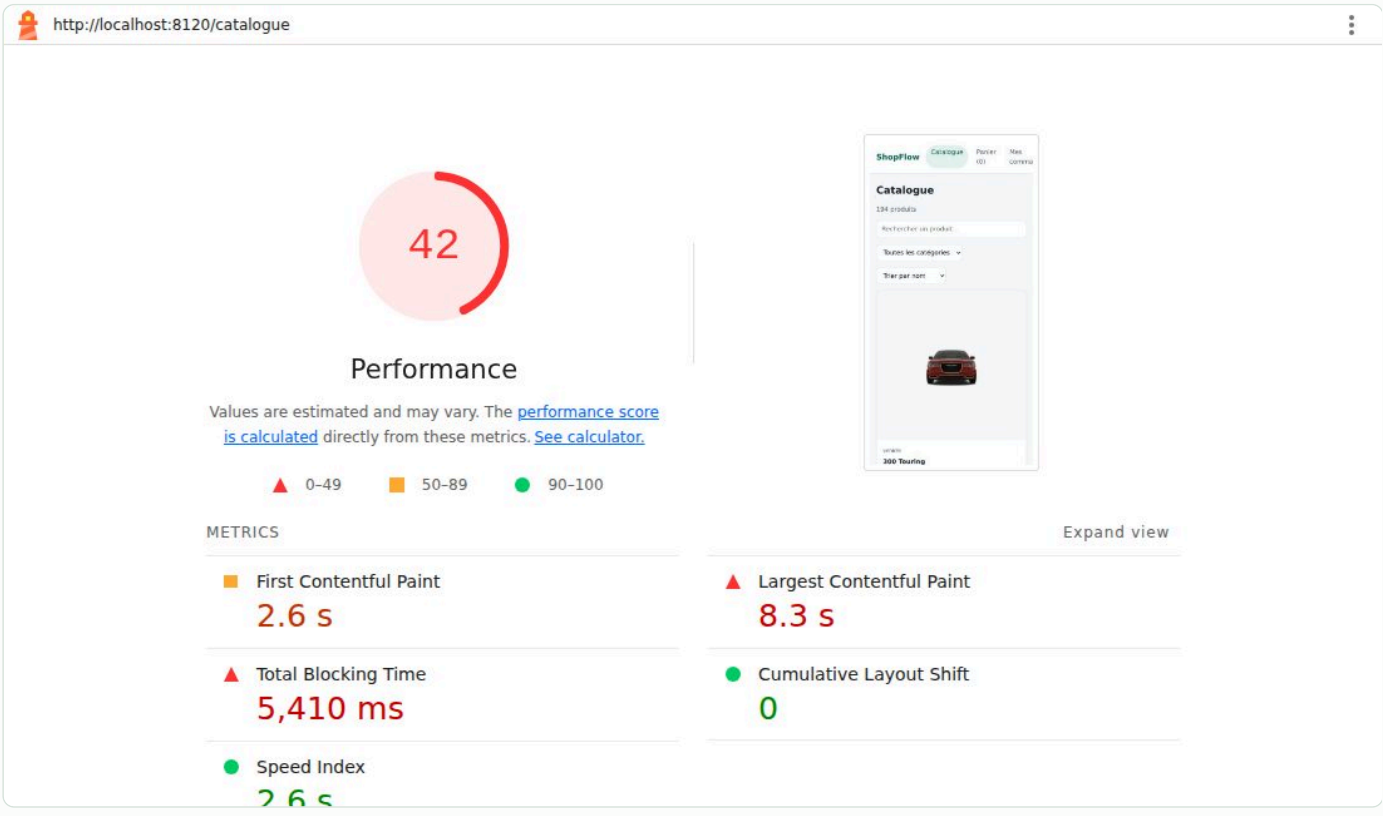
Mesurer une autre page : changer l'adresse dans la commande, par exemple :

```
Terminal 2 — n'importe quel dossier
```

npx lighthouse

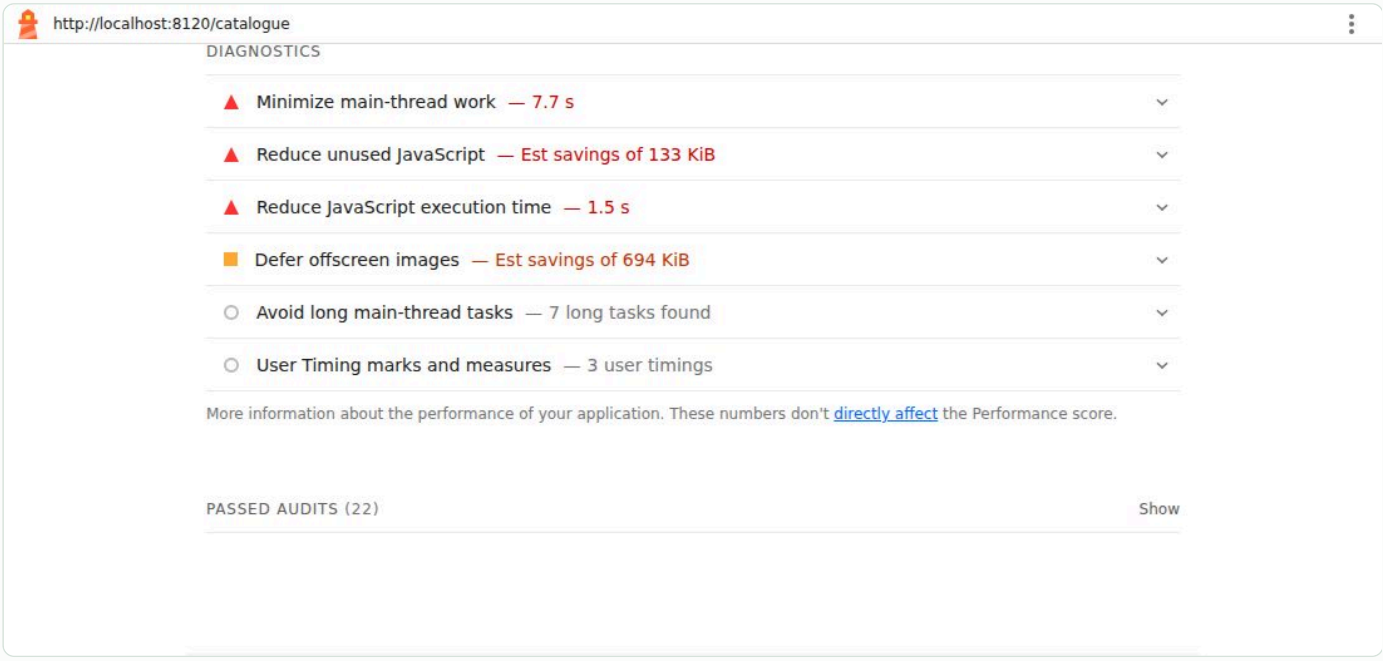
```
npx lighthouse http://127.0.0.1:8080/catalogue/1 --only-categories=performance --form-factor=mobile --view
```

Les pages qui demandent une session (`/mes-commandes` , `/admin`) se mesurent dans le panneau Lighthouse de Chrome, qui garde la connexion : se connecter sur `/connexion` (`emilys` / `emilyspass`), aller sur la page, F12, onglet Lighthouse, **Analyze page load**. La ligne de commande, elle, ouvre un Chrome vierge, sans session.



Lighthouse sur la version de production de ShopFlow : 42 sur 100 en mobile.

Mesure	Ce qu'elle mesure	ShopFlow
First Contentful Paint (FCP)	Premier texte ou première image affichés	2,6 s
Largest Contentful Paint (LCP)	Le plus grand élément visible est affiché	8,3 s
Total Blocking Time (TBT)	Temps pendant lequel la page ne répond pas aux clics	5 410 ms
Cumulative Layout Shift (CLS)	Éléments qui se décalent pendant le chargement	0
Speed Index	Vitesse à laquelle la page se remplit	2,6 s



La partie Diagnostics du même rapport.

Ce que disent les diagnostics :

- **Minimize main-thread work, 7,7 s** et **Reduce JavaScript execution time, 1,5 s** : le navigateur passe son temps à exécuter du JavaScript et à construire la page (2 379 éléments HTML pour les 194 cartes).
- **Defer offscreen images, 694 Kio** : les 194 images sont téléchargées tout de suite, même celles qu'on ne voit pas.
- **Reduce unused JavaScript, 133 Kio** : une partie du code téléchargé ne sert pas sur la page du catalogue.

> **Bon à savoir**
>
> Les chiffres varient d'une mesure à l'autre (43 puis 42 sur deux mesures successives). On fait trois mesures et on garde celle du milieu. Fermer les autres onglets et utiliser une fenêtre de navigation privée évite que les extensions faussent le résultat.

> **Erreur fréquente : CHROME_INTERSTITIAL_ERROR**
>
> Lighthouse s'arrête avec ce message quand la page répond `404`. Deux causes habituelles : une adresse autre que la racine servie sans l'option `-s`, ou le serveur lancé depuis le mauvais dossier (par exemple depuis `dist/shopflow`, il cherche alors `dist/shopflow/dist/shopflow/browser`). Le lancer **depuis la racine du projet**, après avoir vérifié que le build existe :

Terminal 1 — dans shopflow/ (racine du projet, là où se trouve package.json) `ls dist/shopflow/browser/index.html`

```
ls dist/shopflow/browser/index.html
npx serve -s dist/shopflow/browser -l 8080
```

En entreprise : laboratoire, terrain et serveur

Le poste du développeur ne ressemble ni au serveur de production ni aux téléphones des clients, et ce n'est pas grave : on ne mesure pas la même chose au même endroit.

Mesure	Question à laquelle elle répond	Outils
Laboratoire	Ma modification améliore-t-elle ou dégrade-t-elle la page ?	Lighthouse sur le poste ; **Lighthouse CI** dans le pipeline, toujours sur la même machine, avec des budgets qui bloquent une régression ; WebPageTest sur de vrais téléphones d'entrée de gamme
Terrain	Que vivent réellement nos clients ?	**PageSpeed Insights** (données réelles des utilisateurs de Chrome sur 28 jours, pour un site public) ; **Search Console**, rapport « Signaux Web essentiels » ; la bibliothèque **web-vitals** qui envoie LCP, INP et CLS de chaque visiteur
Serveur	Le serveur livre-t-il vite et bien ?	Temps de première réponse (TTFB), compression gzip ou Brotli, en-têtes de cache, HTTP/2, CDN, temps de rendu côté serveur (jour 3) : à vérifier sur la production, dans l'onglet Réseau ou avec PageSpeed Insights

> **À retenir**
>
> Le laboratoire **prouve** qu'une correction marche ; le terrain dit si les clients en **profitent** ; le serveur se mesure **à part**. Le rapport Lighthouse indique la puissance du poste (section « Environment », ligne « CPU/Memory Power ») : sur une machine très puissante, l'option `--throttling.cpuSlowdownMultiplier` renforce le ralentissement pour se rapprocher d'un téléphone moyen.

Lire un score Lighthouse : laboratoire et terrain

Lighthouse est une **mesure de laboratoire** : il simule un téléphone moyen, mais à partir de la puissance réelle du poste. Le même ShopFlow obtient ainsi 42 sur une machine lente et 75 sur un Mac récent ; le temps de blocage (TBT) est la valeur qui varie le plus. La section « Environment » du rapport indique la puissance mesurée (« CPU/Memory Power »).

En entreprise, la question « mon poste ressemble-t-il à la production ? » ne se pose pas ainsi : ce qui compte, ce sont les appareils et les réseaux des **utilisateurs**, car c'est leur navigateur qui exécute Angular. On mesure trois choses, avec trois outils :

Ce qu'on veut savoir	Outil	Ce qu'on regarde
Ce que vivent les vrais utilisateurs (la vérité)	PageSpeed Insights (données réelles de Chrome sur 28 jours), Search Console « Signaux Web essentiels », bibliothèque `web-vitals` dans l'application	LCP, INP, CLS par page et par type d'appareil
Si une modification fait régresser le site	Lighthouse CI dans le pipeline, avec des budgets ; ou PageSpeed Insights, WebPageTest (vrais téléphones, réseau mobile)	Le même test, sur la même machine, à chaque version
Si le serveur répond bien	L'onglet Réseau et PageSpeed Insights sur l'adresse de production	Temps de première réponse (TTFB), compression gzip/Brotli, cache, HTTP/2, CDN, temps du rendu serveur

3. Outil 2 : l’analyse du paquet

Le problème : Lighthouse signale du JavaScript inutile. On veut savoir ce que contient le fichier téléchargé au démarrage.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) · construire avec un fichier de statistiques

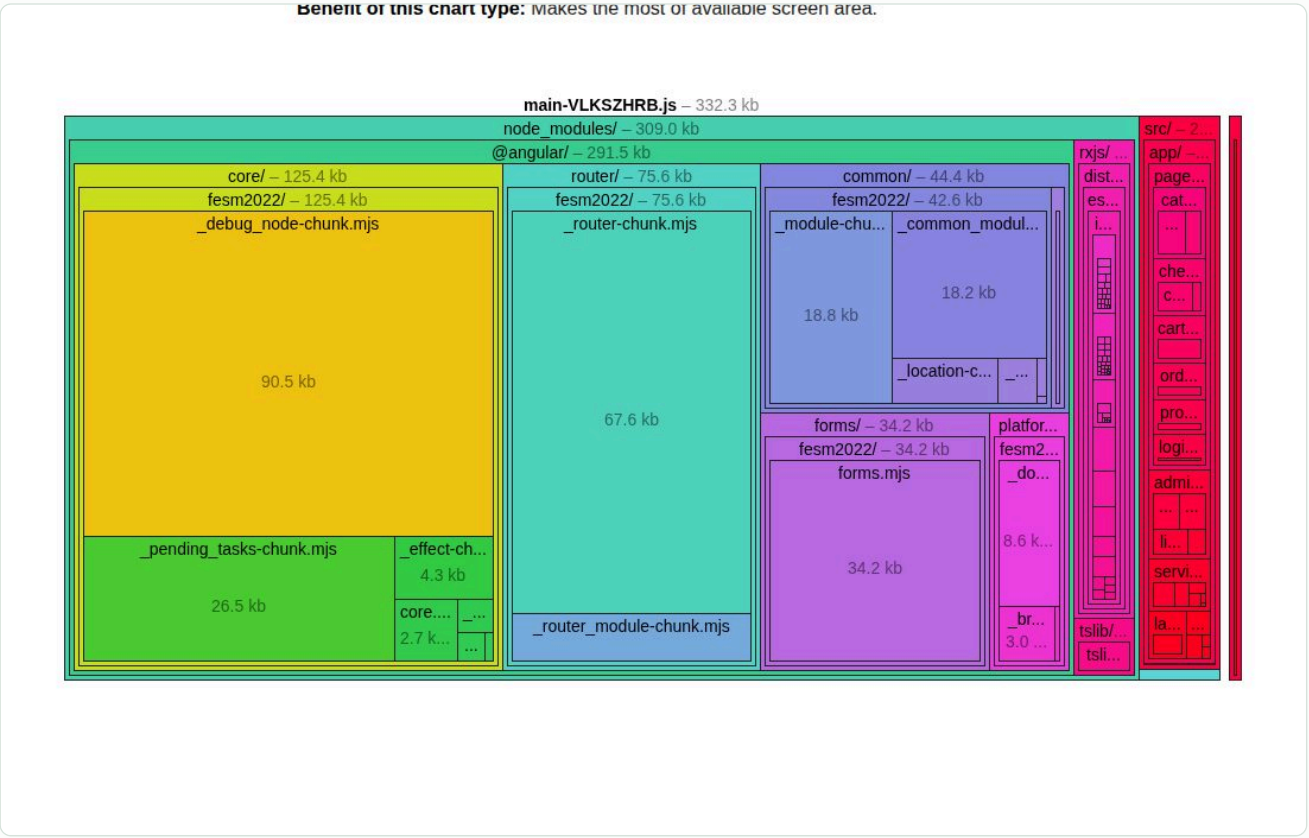
```
npx ng build --stats-json
```

Terminal : shopflow

```
$ npx ng build --stats-json
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-VLKSZHRB.js    | main          | 340.28 kB |               90.81 kB
styles-SZZ3P6BQ.css | styles        |   6.36 kB |                1.60 kB
                    | Initial total | 346.63 kB |               92.41 kB
Application bundle generation complete. [7.477 seconds] - 2026-10-15T14:32:53.831Z
```

Un seul fichier JavaScript, main, de 340 ko (91 ko compressé), et le fichier de statistiques dist/shopflow/browser-stats.json.

On ouvre ensuite `https://esbuild.github.io/analyze/` (l’outil de l’assembleur utilisé par Angular), bouton **Import your metafile**, et on choisit `dist/shopflow/browser-stats.json`. Le fichier ne quitte pas l’ordinateur : l’analyse se fait dans le navigateur.



Le contenu de main : chaque rectangle est un fichier, sa surface est proportionnelle à sa taille.

Contenu de main	Taille	Part
@angular/core	125 ko	38 %
@angular/router	76 ko	23 %
@angular/common	44 ko	13 %
@angular/forms	34 ko	10 %
Code de ShopFlow (`src/`)	23 ko	7 %
rxjs	16 ko	5 %
@angular/platform-browser	12 ko	4 %

Ce qu'on en retient : toutes les pages, administration comprise, sont dans ce fichier. `@angular/forms` (34 ko) n'est utilisé que par la connexion et la commande, mais il est téléchargé par tous les visiteurs du catalogue. C'est ce que corrige le chargement à la demande (séquence 5).

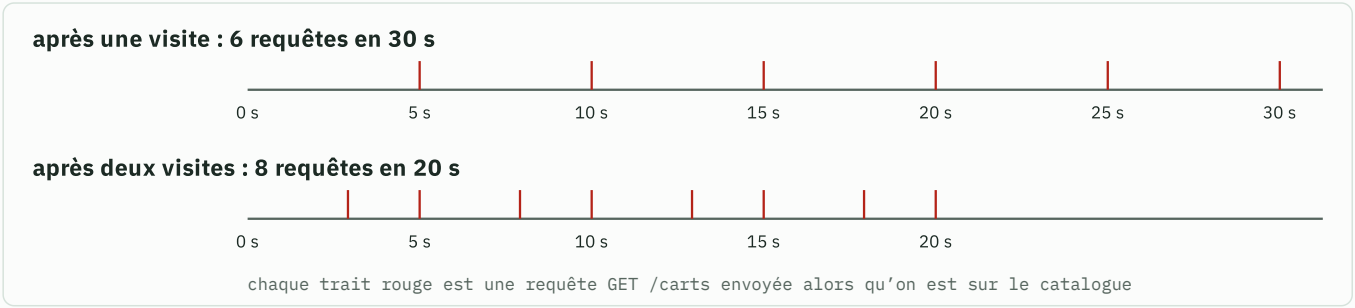
4. Outil 3 : l'onglet Réseau

Le problème : on veut savoir quelles requêtes l'application envoie, et quand.

Dans Chrome : F12, onglet **Réseau**, cocher **Conserver le journal**, taper `dummyjson` dans le filtre. Puis, sans recharger la page, suivre ce parcours en cliquant dans le menu :

1. Catalogue : compter les requêtes `products?limit=0`.
2. Administration, puis État du stock : recompter.
3. Revenir au catalogue et attendre 30 secondes en regardant les requêtes `carts`.
4. Retourner dans l'administration, revenir au catalogue, attendre encore.

Observation	Mesure sur ShopFlow
Requêtes au chargement du catalogue	199, dont 194 images
Catalogue complet demandé à l'API	3 fois (catalogue, tableau de bord, stock)
Requêtes `carts` dans les 30 s qui suivent la sortie de l'administration	6, une toutes les 5 s
Après une deuxième visite de l'administration	8 requêtes en 20 s : deux minuteries tournent



Les minuteries de l'administration continuent après qu'on l'a quittée, et chaque visite en ajoute une.

5. Outil 4 : Angular DevTools et le comptage des appels

Le problème : Lighthouse dit que le navigateur exécute trop de JavaScript. On veut savoir quels composants Angular travaillent, et combien de fois.

Pour avoir un chiffre exact, on ajoute temporairement un compteur dans les deux méthodes appelées par les templates :

src/app/pages/catalogue/catalogue.ts (extrait, ajout temporaire) (extrait) protected produitsAffiches(produits:

```
  protected produitsAffiches(produits: Product[]): Product[] {
    console.count('produitsAffiches');
    const terme = this.recherche.trim().toLowerCase();
```

src/app/pages/catalogue/product-card.ts (extrait, ajout temporaire) (extrait) protected prixFinal(): number {

```
  protected prixFinal(): number {
    console.count('prixFinal');
```

Moment	produitsAffiches()	prixFinal()
Affichage du catalogue	2 appels	194 appels
Saisie de `phone` (5 touches)	5 appels	88 appels

À chaque touche, la liste des 194 produits est refiltrée et retriée, sans attendre la fin de la saisie. Avec `track $index`, Angular identifie les cartes par leur position : quand le filtre change, les cartes gardent leur place mais reçoivent un autre produit, et sont redessinées. Les composants en OnPush limitent déjà les dégâts : sans eux, les 194 cartes seraient vérifiées à chaque touche.

6. Outil 5 : ce que voient un moteur de recherche, les tests et le reste

Ce que reçoit un moteur de recherche : on demande la page au serveur sans navigateur, comme le fait un robot d'indexation.

Terminal : shopflow (ng serve lancé dans un autre terminal)

```
$ curl -s http://localhost:4200/catalogue | grep -n "<body\|app-root\|</body"
12:  <body>
13:    <app-root></app-root>
14:  <script src="main.js" type="module"></script></body>
```

Le serveur renvoie une balise <app-root> vide : aucun produit n'est visible sans exécuter le JavaScript.

Les tests :

Terminal : shopflow

```
$ npx ng test --watch=false
Chrome Headless 155.0.8059.79 (Linux 0.0.0): Executed 0 of 0 SUCCESS (0.005 secs / 0 secs)
TOTAL: 0 SUCCESS
```

Aucun test : « Executed 0 of 0 ».

Les autres constats, faits pendant la visite :

- `/admin` et `/mes-commandes` s'ouvrent sans être connecté.
- Le formulaire de commande accepte un envoi vide.
- Le prix de la fiche produit diffère de celui de la carte.
- Tous les textes sont écrits en français dans les templates : pas de version anglaise possible.
- Aucune documentation du code.
- Tout le code est dans un seul projet : la carte produit, qui pourrait servir ailleurs, dépend directement du panier.

7. TP en binômes : compléter le diagnostic (15 min)

Énoncé à distribuer

Chaque binôme refait les mesures sur son poste et remplit la grille ci-dessous. Une ligne par problème. La colonne « Besoin » se formule comme une exigence vérifiable, pas comme une solution technique.

diagnostic-shopflow.md

```
# Diagnostic de ShopFlow

| N° | Constat | Outil et mesure | Gravité (1 à 3) | Besoin (exigence vérifiable) |
|----|---------|-----------------|-----------------|------------------------------|
| 1  |         |                 |                 |                              |
| 2  |         |                 |                 |                              |

Gravité : 1 = gêne, 2 = dégrade l’usage ou la performance, 3 = faille ou erreur visible du client.
```

8. Corrigé : le cahier des charges de ShopFlow

On remplit le tableau au vidéoprojecteur à partir des grilles des binômes. Il reste affiché ou imprimé pendant les trois jours : chaque séquence commence par la ligne qu’elle traite et se termine par la mesure qui la valide.

▶ **Afficher le cahier des charges attendu**

12 h 30 – 13 h 30 : déjeuner

14 h 30 – 15 h 15

Séquence 3 : Nx dans ShopFlow (première partie)

> **Au programme de cette séquence (45 min)**
>
> **Point du programme** : module 1, « Introduction au CLI Nx ».
>
> **Objectif** : les apprenants savent dire à quoi sert Nx, l'installent dans ShopFlow, lancent les tâches en ligne de commande et dans Nx Console, et savent où cliquer.
>
> - 25 min : à quoi sert Nx, concrètement
> - 20 min : installation dans ShopFlow

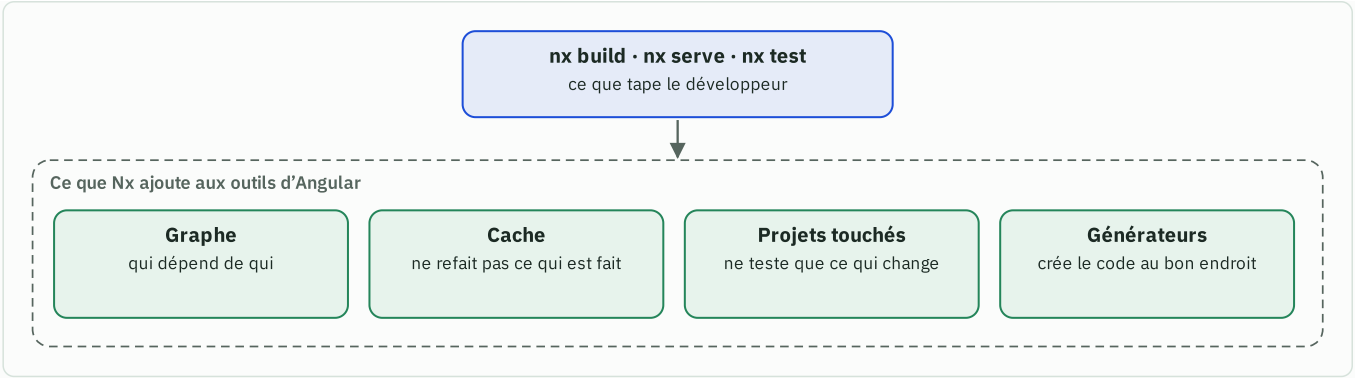
1. À quoi sert Nx, concrètement

Le problème. Tant qu'une application est petite, Angular CLI suffit. Mais ShopFlow grossit : demain, l'entreprise veut une **seconde application** (un back-office pour les vendeurs) qui affiche les mêmes cartes produit et utilise les mêmes services. Trois difficultés apparaissent :

- **Partager du code** sans le copier-coller d'une application à l'autre.
- **Savoir ce que casse une modification** : si je change la carte produit, quelles applications faut-il retester ?
- **Ne pas tout reconstruire** à chaque petite modification : sur un gros projet d'entreprise, une construction complète prend plusieurs minutes, et l'intégration continue relance tout à chaque commit.

Nx est l'outil qui organise un dépôt en **plusieurs petits projets** (applications et bibliothèques) et qui **ne refait que ce qui est nécessaire**. Il ne remplace pas Angular : il lance les mêmes outils (`@angular/build`), et ajoute autour.

Situation	Avec Angular CLI seul	Avec Nx
Réutiliser la carte produit dans une seconde application	Copier le fichier ; les deux copies divergent	Une bibliothèque `@shopflow/ui`, importée par les deux applications (séquence 4)
Savoir ce que touche une modification	Chercher à la main	`nx graph` dessine qui dépend de qui ; `nx affected` liste les projets touchés
Reconstruire sans rien changer	Tout est refait : 7,8 s pour ShopFlow	Résultat relu dans le cache : 11 ms
Tests dans l'intégration continue	Tous les tests, à chaque fois	Seulement ceux des projets touchés
Créer un composant au bon endroit	À la main, chacun à sa façon	Générateurs (`nx g`) ou formulaire de Nx Console : mêmes conventions pour toute l'équipe



Nx lance les outils d'Angular et ajoute quatre services : le graphe, le cache, le calcul des projets touchés et les générateurs.

2. Installer Nx dans ShopFlow

ShopFlow est un projet Angular CLI. On ne recrée rien : `nx init` le transforme sur place.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx@latest init

```
npx nx@latest init
```

La commande installe les paquets Nx et convertit la configuration. Elle pose quelques questions : on accepte les réglages proposés et on répond **non** au cache distant (Nx Cloud), inutile en formation. À la première commande Nx suivante, la question « Share usage data with the Nx team? » se règle avec les flèches puis Entrée ; elle ne revient plus.

Terminal : shopflow

```
$ npx nx@latest init
 NX   ✅ The Angular version is compatible with the latest version of Nx!
 NX   🐳 Nx initialization
 NX   📦 Installing dependencies
 NX   📝 Setting up workspace
 NX   🎉 Done!
$ git status --short
 M .gitignore
 M .vscode/extensions.json
 D angular.json
 M package-lock.json
 M package.json
?? nx.json
?? project.json
```

nx init dans ShopFlow, puis la liste des fichiers modifiés.

Fichier	Ce qui a changé
`angular.json`	Supprimé : la configuration passe dans `project.json`
`project.json`	Créé : les mêmes tâches (`build` , `serve` , `test`), avec les mêmes outils Angular
`nx.json`	Créé : réglages communs à tous les projets (cache, ordre des tâches)
`package.json`	Scripts `ng …` devenus `nx …` ; paquets `nx` et `@nx/angular` ajoutés
`.gitignore`	Ajout de `.nx/cache` : le cache reste sur le poste

project.json (extrait) (extrait)

```
{
  "name": "shopflow",
  "projectType": "application",
  "targets": {
    "build": { "executor": "@angular/build:application" },
    "serve": { "executor": "@angular/build:dev-server" }
  }
}
```

15 h 15 – 15 h 30 : pause

15 h 30 – 15 h 40

TP final : remettre Cuisto en état

Le TP final en bref

Ce qu'on vous remet : Cuisto, une application de recettes de cuisine en Angular 22 (le dépôt Cuisto fourni par le formateur). Elle fonctionne, mais elle a été écrite vite : elle est lente, mal organisée, peu sûre, non testée, et contient au moins deux défauts visibles par l'utilisateur.

Ce qu'on attend de vous : la même démarche qu'avec ShopFlow, en autonomie : mesurer, rédiger un cahier des charges, corriger module par module, re-mesurer. Les problèmes ressemblent à ceux de ShopFlow, mais leur code et leurs corrections sont différents.

Organisation : seul ou en binôme, pendant la formation : chaque jour de 15 h 30 à 17 h 00, sur les modules du jour (énoncés « TP Cuisto, jour 1, 2, 3 »). Remise le jour 3 à 16 h 45.

1. L'application

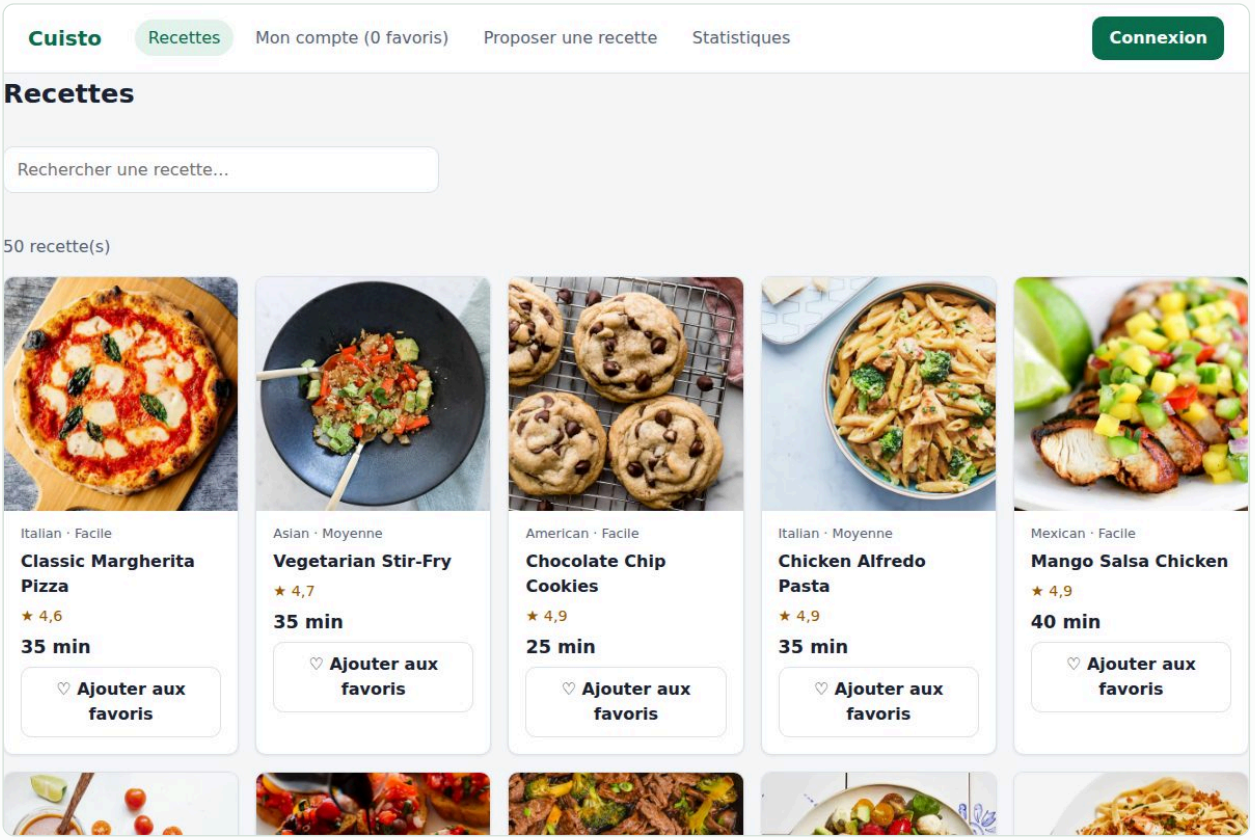
Cuisto utilise l'API publique DummyJSON (50 recettes). Compte de démonstration : `emilys` / `emilyspass` (administrateur) ; `oliviaw` / `oliviawpass` (modérateur).

Terminal 2 — dans le dossier qui contient formation-angular/ · seulement si Cuisto n'a pas été cloné à l'introduction

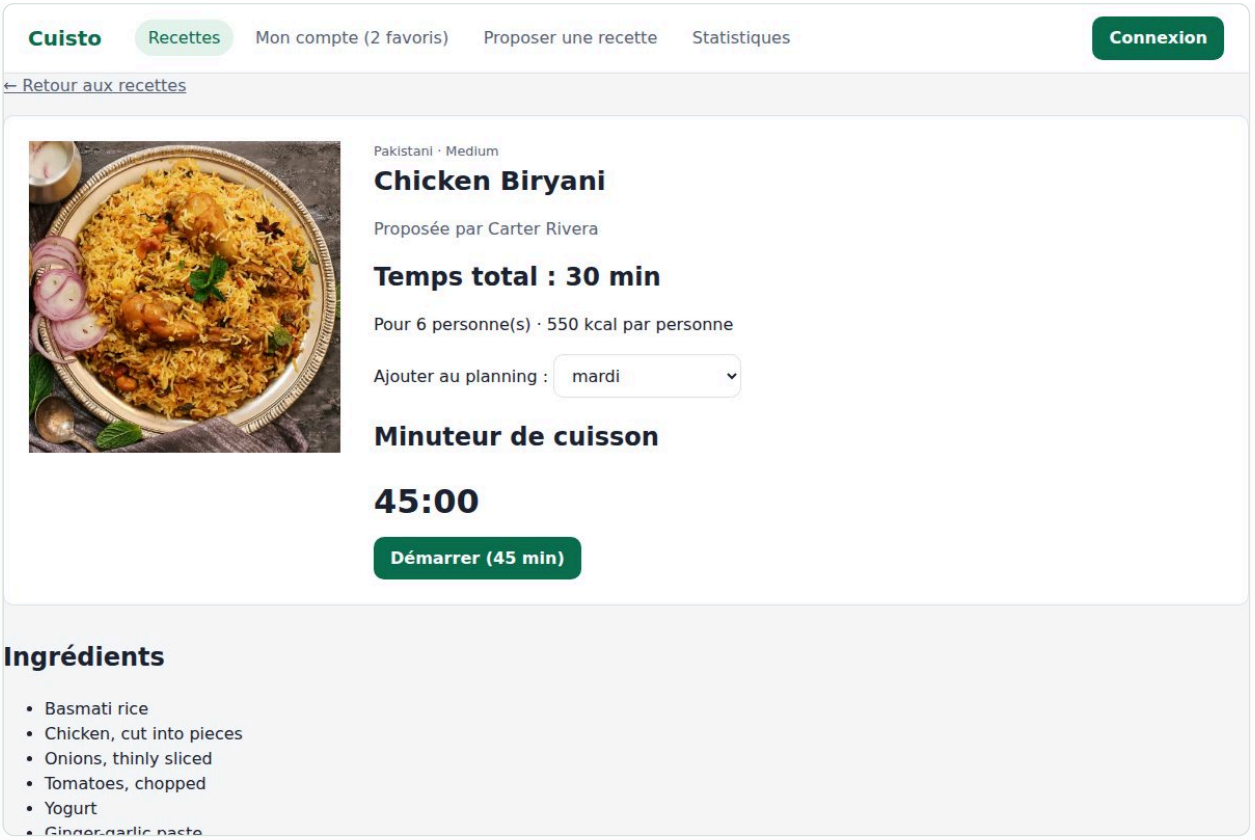
```
cd formation-angular
```

```
cd formation-angular
git clone https://github.com/adamfrance/cours-angular-avance-cuisto-app-exercices.git cuisto
cd cuisto
npm install
npm start -- --port 4201
```

Page	Adresse	Rôle
Recettes	`/recettes`	Liste, recherche (faite par le serveur), favoris
Fiche recette	`/recettes/:id`	Ingrédients, étapes, auteur, minuteur de cuisson, ajout au planning, recettes similaires
Mon compte	`/compte/favoris`, `/compte/planning`, `/compte/mes-recettes`	Favoris, planning de la semaine avec liste de courses, recettes publiées
Connexion	`/connexion`	Connexion par l'API
Proposer une recette	`/proposer`	Formulaire de proposition
Statistiques	`/admin`	Indicateurs sur les recettes



La liste des recettes.



Une fiche recette, avec son minuteur de cuisson.

2. Ce que vous devez faire

Étape	Attendu
1. Diagnostic	Mesures de départ avec au moins quatre outils (Lighthouse en version de production, analyse du paquet, onglet Réseau, Angular DevTools ou comptage des appels, console). Chaque mesure : outil, page, chiffre.
2. Cahier des charges	Une ligne par problème : constat mesuré, besoin formulé comme une exigence vérifiable, critère de validation, module concerné.
3. Module 1	Nx installé, découpage en bibliothèques, pages chargées à la demande, `@defer`, images optimisées, plus de méthodes dans les templates, `track` par identité, documentation Compodoc.
4. Module 2	Flux RxJS sans abonnements imbriqués, recette chargée une seule fois, recherche en fin de saisie, minuteries arrêtées, pages privées protégées (`CanActivateFn`, `CanActivateChildFn`), fiche chargée par un resolver.
5. Module 3	État des favoris en signaux (`signal`, `asReadonly`, `computed`, `effect`), ponts avec RxJS là où c'est utile.
6. Module 4	Configuration regroupée dans une fonction `provideXxx()`, au moins un composant standalone réutilisable dans une bibliothèque `ui`.
7. Module 5	Interface en français et en anglais, avec identifiants `@@` et pluriels ICU.
8. Module 6	Formulaire de proposition réactif et typé, validation et messages d'erreur, ingrédients en `FormArray`.
9. Module 7	Tests Karma et Jasmine (pipe, service, guard, composant), au moins un défaut corrigé en TDD (test rouge, puis vert), rapport de couverture, au moins deux parcours Playwright.
10. Module 8	Rendu côté serveur avec `@angular/ssr` : modes de rendu par route justifiés, liste des recettes lisible sans JavaScript.
11. Bilan	Les mesures de l'étape 1 refaites avec les mêmes outils, et l'état de chaque ligne du cahier des charges.

Livrables

- Le dépôt Git du projet, avec **une étiquette par étape** (`etape-01-diagnostic` , `etape-03-module1` …) pour que l'évaluateur puisse voir chaque état.
- Un fichier `RAPPORT.md` à la racine : diagnostic, cahier des charges, choix faits (et ceux écartés), mesures avant et après, difficultés rencontrées.
- La commande pour lancer chaque élément : application, tests, tests de bout en bout, documentation, version anglaise, serveur SSR.

Règles

Angular 22 et l'écriture actuelle partout (`@if` , `@for` , `inject()` , composants standalone, guards en fonctions). Toute affirmation du rapport (« plus rapide », « plus de requêtes ») est appuyée par une mesure. Une console sans erreur est exigée.

3. Grille d'évaluation (sur 20)

Critère	Points	Ce qui est vérifié (étapes indispensables)
Diagnostic et rapport	3	Mesures avant/après chiffrées ; défauts listés ; « où je me suis arrêté » renseigné
Module 1 : Nx et bibliothèques	4	Nx installé ; trois bibliothèques avec leurs alias ; l'application se construit
Module 2 : RxJS et guards	3	Recherche avec `debounceTime` et `switchMap` ; pages du compte protégées
Module 3 : signaux	2	Planning dans un signal ; nombre de repas en `computed`
Module 5 : internationalisation	2	Cinq textes traduits ; deux versions construites
Module 6 : formulaires	2	Formulaire réactif ; nom obligatoire ; envoi impossible tant qu'il est invalide
Module 7 : tests	2	Un test rouge puis vert sur la durée de la fiche
Historique Git	2	Étiquettes `jour-1` , `jour-2` , `jour-3`

Bonus (jusqu'à 4 points, dans la limite de 20) : les étapes « si le temps le permet » réalisées et vérifiables — carte sans service, chargement à la demande, templates sans méthodes, `provideCuisto()` , `FormArray` , Playwright, rendu côté serveur…

15 h 40 – 17 h 00

TP Cuisto, jour 1

Objectif du TP

À la fin du TP : Cuisto est mesuré, géré par Nx et découpé en trois bibliothèques (types, services, composants d'affichage), comme ShopFlow en cours.

Pourquoi : on ne corrige bien que ce qu'on a mesuré ; les bibliothèques permettent de réutiliser le code et de savoir ce que touche une modification.

À rendre : le dépôt avec l'étiquette `jour-1` et le fichier `RAPPORT.md` (mesures, défauts trouvés, endroit où vous vous êtes arrêté).

Copier les commandes depuis la version HTML de ce document (bouton **Copier**) ou depuis le fichier de commandes, jamais depuis le PDF. Sous Windows, utiliser **Git Bash**.

1. Lancer Cuisto et créer le rapport (5 min)

À obtenir : Cuisto tourne sur le port 4201 et le fichier `RAPPORT.md` existe.

Terminal 1 — dans le dossier qui contient formation-angular/ `cd formation-angular/cuisto`

```
cd formation-angular/cuisto
npm start -- --port 4201
```

cuisto/RAPPORT.md (à créer) `# Rapport Cuisto`

```
# Rapport Cuisto

## Jour 1
| Mesure                 | Avant | Après |
|------------------------|-------|-------|
| Score Lighthouse       |       |       |
| Poids transféré        |       |       |
| Images à l'ouverture   |       |       |

Défauts trouvés :
-

Où je me suis arrêté :
-
```

2. Mesurer (20 min)

À obtenir : le tableau « Avant » rempli et au moins cinq défauts notés.

Terminal 2 — dans cuisto/ (racine du projet, là où se trouve package.json) `npx ng build`

```
npx ng build
npx serve -s dist/cuisto/browser -l 8081
```

Terminal 3 — n'importe quel dossier `npx lighthouse`

```
npx lighthouse http://127.0.0.1:8081/recettes \
  --only-categories=performance --form-factor=mobile \
  --view
```

Lancer Lighthouse trois fois et garder la valeur du milieu. Puis, dans Chrome (F12, onglet **Réseau**, **Désactiver le cache**, recharger), relever le poids transféré et le nombre d'images.

Défauts à chercher : trop d'images téléchargées à l'ouverture ; requête des « Recettes similaires » envoyée avant qu'on voie le bloc ; minuteur qui ne bouge pas ; carte de recette qui navigue et gère les favoris elle-même ; méthodes appelées dans les templates ; tout le code dans un seul fichier.

3. Installer Nx (10 min)

À obtenir : Nx installé ; la deuxième construction est relue dans le cache.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `npx nx@latest init`

```
npx nx@latest init
npx nx build cuisto
npx nx build cuisto
```

À la question sur Nx Cloud, répondre non. Dans Nx Console, le même résultat s'obtient avec : icône **Nx** > **Projects** > **cuisto** > **build** > ▷.

> **Vérifier** : la deuxième construction affiche « read the output from the cache ».

4. Créer les bibliothèques (30 min)

À obtenir : trois bibliothèques `@cuisto/models`, `@cuisto/data-access` et `@cuisto/ui`, utilisées par l'application, qui se construit sans erreur.

a. Créer les bibliothèques, en ligne de commande **ou** dans Nx Console (pas les deux) :

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `npx nx g @nx/angular:library`

```
npx nx g @nx/angular:library --directory=libs/recettes/models \
  --name=recettes-models --importPath=@cuisto/models \
  --unitTestRunner=none --prefix=cu \
  --skipModule --tags=type:models
npx nx g @nx/angular:library --directory=libs/recettes/data-access \
  --name=recettes-data-access --importPath=@cuisto/data-access \
  --unitTestRunner=none --prefix=cu \
  --skipModule --tags=type:data-access
npx nx g @nx/angular:library --directory=libs/recettes/ui \
  --name=recettes-ui --importPath=@cuisto/ui \
  --unitTestRunner=none --prefix=cu \
  --skipModule --tags=type:ui
```

Dans Nx Console : `Cmd+Maj+P` (`Ctrl+Maj+P`) > **Nx: Generate (UI)** > **@nx/angular - library**, avec les mêmes valeurs ; **unitTestRunner**, **prefix**, **tags** et **skipModule** sont dans **Show all options**.

b. Y déplacer les fichiers (après avoir supprimé les exemples créés par le générateur) :

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `rm -rf libs/recettes/models/src/lib/*`

```
rm -rf libs/recettes/models/src/lib/* libs/recettes/data-access/src/lib/* \
  libs/recettes/ui/src/lib/*
mkdir -p libs/recettes/ui/src/lib/recipe-card
git mv src/app/models/* libs/recettes/models/src/lib/
git mv src/app/services/* libs/recettes/data-access/src/lib/
git mv src/app/pages/recipes/recipe-card.ts \
  src/app/pages/recipes/recipe-card.html \
  libs/recettes/ui/src/lib/recipe-card/
```

c. Déclarer ce que chaque bibliothèque exporte, en remplaçant le contenu de ses fichiers `index.ts` :

`libs/recettes/models/src/index.ts`

```
export * from './lib/recipe';
export * from './lib/user';
export * from './lib/planning';
```

`libs/recettes/data-access/src/index.ts`

```
export * from './lib/recipe-service';
export * from './lib/favorites-service';
export * from './lib/planning-service';
export * from './lib/auth-service';
export * from './lib/auth-interceptor';
```

`libs/recettes/ui/src/index.ts`

```
export * from './lib/recipe-card/recipe-card';
```

d. Corriger les imports : dans VS Code, `Cmd+Maj+H` (`Ctrl+Maj+H`), activer l'icône `.*`, écrire `src, libs` dans « fichiers à inclure », puis faire les trois remplacements ci-dessous (bouton **Remplacer tout**).

`Les trois remplacements, à copier dans VS Code`

```
Rechercher 1   : from '(\.\./|\./)+models/[a-z-]+'
Remplacer par  : from '@cuisto/models'

Rechercher 2   : from '(\.\./|\./)+services/[a-z-]+'
Remplacer par  : from '@cuisto/data-access'

Rechercher 3   : from '(\.\./|\./)+(pages/recipes/)?recipe-card'
Remplacer par  : from '@cuisto/ui'
```

`Terminal — dans cuisto/ (racine du projet, là où se trouve package.json)`

```
npx nx build cuisto
```

> **Vérifier** : la construction se termine par « Successfully ran target build ». Un message « Could not resolve "../models/..." » signale un import qui n'a pas été remplacé.

5. Mesurer à nouveau et rendre (10 min)

À obtenir : la colonne « Après » remplie et le dépôt étiqueté.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) git add -A

```
git add -A
git commit -m "Jour 1"
git tag jour-1
git push --follow-tags
```

Pour aller plus loin (facultatif)

À faire seulement si les étapes précédentes sont terminées. Chaque point donne des points bonus.

- **Rendre la carte indépendante** : la carte ne doit plus injecter de service. Elle reçoit « est-elle en favori ? » par une entrée et signale « ouvrir » et « changer le favori » par des sorties ; ce sont les pages qui naviguent et gèrent les favoris.
- **Charger à la demande** : chaque page dans un fichier séparé (`loadComponent`) ; « Recettes similaires » dans un bloc `@defer (on viewport)` dont la requête ne part qu'à l'affichage ; images avec `ngSrc` .
- **Alléger les templates** : remplacer les méthodes appelées dans les templates par des pipes purs ; faire défiler le minuteur avec `ChangeDetectorRef` .
- **Documenter** : générer la documentation avec Compodoc.